



南开大学

Nankai University

软件学院

Online-Boutique 微服务测试与维护大作业 实验报告

基于 Online-Boutique 的部署、扩展、监控、故障注入、测试与智能运维实践

学 院：软件学院

成 员：王秀强 邱俊杰 韦厚林 任泓旭 邓锦尧 段坤良

项目仓库：<https://github.com/Affordan/online-boutique-testing-maintenance>

2025~2026 第二学期

使用 L^AT_EX 撰写于 2026 年 6 月 11 日

目 录

1	大作业概述	4
1.1	项目背景与选题说明	4
1.2	项目完成范围	4
1.3	课程要求覆盖情况	6
2	项目组织、环境与小组分工	6
2.1	技术组成	6
2.2	实验环境	7
2.3	仓库目录结构	8
2.4	小组分工与个人任务	9
3	Online-Boutique 主系统部署	11
3.1	部署目标	11
3.2	部署命令与执行说明	11
3.3	部署结果验证	12
4	新增微服务开发与接入	15
4.1	新增服务设计目标	15
4.2	coupon-service 实现	15
4.3	inventory-service 实现	16
4.4	容器构建、部署与环境变量注入	17
4.5	新增服务验证结果	17
5	Prometheus 与 Grafana 监控实验	19
5.1	监控目标	19
5.2	监控组件部署	19
5.3	指标接入设计	19
5.4	监控结果展示	21

6	ChaosMesh 故障注入实验	23
6.1	故障注入目标	23
6.2	ChaosMesh 安装与验证	23
6.3	故障实验设计	24
6.4	代表性实验结果	25
7	Selenium 功能测试与 JMeter 性能测试	27
7.1	测试目标	27
7.2	Selenium 功能测试	27
7.3	JMeter 性能测试设计	29
7.4	JMeter 性能测试结果	30
8	异常数据集整理与算法分析	32
8.1	数据来源与字段设计	32
8.2	数据集规模与质量校验	33
8.3	指标波动分析	33
8.4	论文算法复现实验设计	35
8.5	DA-VAE 异常检测复现	36
8.6	KPIRoot 根因定位复现	38
8.7	算法效果评估与局限	42
9	智能运维 Agent 实现	43
9.1	实现目标与安全边界	43
9.2	Agent 总体结构	44
9.3	异常分析与根因分类	45
9.4	诊断报告输出	46
9.5	闭环运维展示	47
10	项目结果分析与问题处理	47
10.1	完成结果汇总	47
10.2	实验中的主要问题与处理	48
11	总结	49
1	附录：主要命令汇总	49
1.1	主系统部署命令	49
1.2	新增服务命令	49

1.3	监控与故障命令	50
1.4	数据、算法与 Agent 命令	50
2	附录：提交材料清单	50

1 大作业概述

1.1 项目背景与选题说明

本次大作业围绕微服务系统的部署、测试与维护展开。小组选择 Online-Boutique 作为实验对象。Online-Boutique 是一个典型在线商店微服务系统，包含前端、商品目录、购物车、结算、支付、推荐、邮件、广告、负载生成等服务。该系统服务数量适中，调用关系清楚，业务链条完整，适合开展本地 Kubernetes 部署、服务扩展、监控采集、故障注入、自动化测试、性能测试、异常检测 and 智能运维实验。

相较于只完成给定 SockShop 部署的基础要求，本项目选择了更复杂的开源微服务系统，并在原系统基础上新增两个业务微服务：coupon-service 和 inventory-service。新增服务完成 FastAPI 源码、Dockerfile、Kubernetes Deployment、Service、接口验证和 Prometheus 指标暴露，符合课程第三档中“在复杂开源微服务系统基础上完成一到两个微服务开发”的要求。项目还实现了轻量级智能运维 Agent，用于读取监控数据、识别异常服务、完成根因分类并生成诊断报告，作为智能运维加分项。

1.2 项目完成范围

本项目最终围绕 Online-Boutique 完成了以下工作：

- 在本地 Windows 环境中使用 Docker、Minikube、kubectl 和 Helm 部署 Online-Boutique。
- 开发并部署 coupon-service 优惠券服务和 inventory-service 库存服务。
- 通过环境变量把新增服务地址注入 frontend、checkoutservice 和 productcatalogservice。
- 使用 Prometheus 和 Grafana 采集并展示 Pod 状态、CPU、内存、请求量、错误率、延迟、网络流量等指标。
- 使用 ChaosMesh 对系统注入 Pod Kill、CPU 压力、内存压力、网络延迟、网络丢包等故障。
- 使用 Selenium 进行主业务链功能测试，使用 JMeter 完成 10、30、50、100 用户并发性能测试。
- 整理 normal_metrics.csv、fault_metrics.csv 和 all_metrics_labeled.csv，形成可用于异常检测研究的数据集。
- 基于数据集完成异常检测分析准备，并实现 AIOps Agent 诊断报告生成。
- 整理 GitHub 仓库、报告 PDF 和展示 PPT 材料。

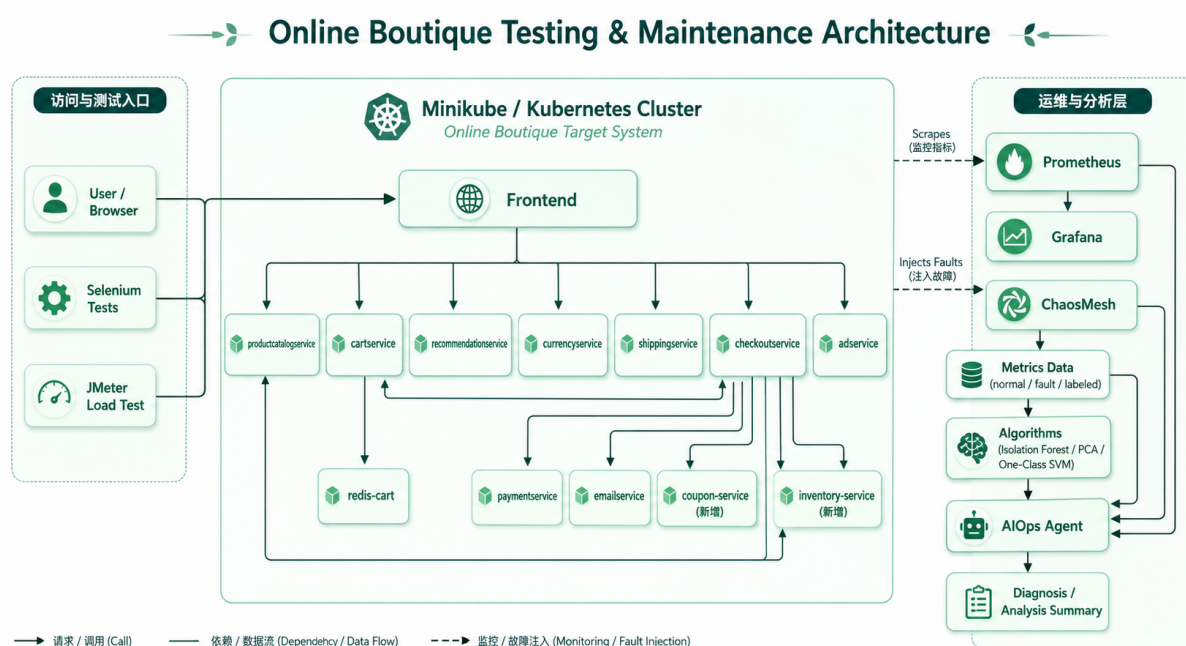


图 1: Online-Boutique 测试与维护项目总体架构图

图 1 展示了本项目的总体结构。底部是本地 Kubernetes 与 Online-Boutique 微服务系统，系统侧扩展了优惠券和库存两个业务服务；中间是 Prometheus、Grafana、ChaosMesh、Selenium 和 JMeter 构成的测试维护支撑环境；上层是数据集、算法分析和 AIOps Agent。这样的组织方式使部署、测试、监控、故障、数据和智能诊断保持统一。

1.3 课程要求覆盖情况

表 1: 课程要求与项目完成内容对应关系

课程要求	本项目完成内容	完成状态
微服务系统部署	使用 Minikube 部署 Online-Boutique, 验证 Pod、Service 和前端页面	已完成
复杂微服务系统	选择 Online-Boutique, 而非仅使用基础 SockShop	已完成
新增微服务开发	新增 coupon-service 与 inventory-service 两个 FastAPI 服务	已完成
监控与维护	部署 Prometheus 与 Grafana, 接入 Kubernetes 指标与新增服务指标	已完成
故障注入	使用 ChaosMesh 进行 Pod Kill、CPU 压力、内存压力、网络延迟、网络丢包实验	已完成
功能测试	使用 Selenium 覆盖首页、商品详情、购物车、结账等主业务链	已完成
性能测试	使用 JMeter 完成 10/30/50/100 用户并发测试	已完成
异常数据采集	生成 normal、fault、all 三类 CSV 数据, 并完成统计校验	已完成
论文算法复现	基于指标数据完成异常检测数据准备与算法分析材料	已完成
智能运维加分项	实现 AIOps Agent 的异常检测、根因分类和诊断报告生成	已完成

2 项目组织、环境与小组分工

2.1 技术组成

本项目采用本地 Kubernetes 环境运行, 不依赖 Google Cloud。这样可以降低账号、计费、权限和网络条件带来的不确定性。项目技术组成见表 2。

表 2: 项目技术组成

类型	工具或技术	用途
目标系统	Online-Boutique	作为被部署、测试和维护的微服务系统
新增微服务	FastAPI	实现优惠券服务和库存服务
容器运行	Docker Desktop	构建镜像、导入 Minikube、运行容器
本地集群	Minikube	在本地启动 Kubernetes 集群
集群管理	kubectl	部署资源、查看 Pod、Service、日志和事件
包管理	Helm	安装 Prometheus/Grafana 与 ChaosMesh
监控采集	Prometheus	采集 Pod、资源与应用级指标
可视化	Grafana	展示系统监控看板和故障对比图
故障实验	ChaosMesh	注入 Pod Kill、CPU、内存、网络类故障
功能测试	Selenium	模拟用户访问、加购和结账行为
性能测试	JMeter	构造并发请求并输出性能指标
数据分析	Python	处理监控 CSV、统计指标波动、支撑异常检测
智能运维	AIOps Agent	读取数据、识别异常、分类根因、输出诊断建议

2.2 实验环境

项目推荐在 Windows 10/11 上运行，主要工具包括 Docker Desktop、kubectl、Minikube、Helm、Git、Python 3.10+、Chrome、Selenium 和 JMeter。仓库默认使用 PowerShell 脚本，避免在 Windows 中误进入 WSL 环境导致 Minikube、Helm、Docker 命令不可见。

```
1 docker --version
2 kubectl version --client
3 minikube version
4 helm version
5 git --version
6 python --version
```

Listing 1: 环境检查命令

本项目统一使用以下运行约定：

表 3: 项目运行约定

项目	取值
Minikube profile	online-boutique-lab
Kubernetes namespace	online-boutique
Online-Boutique 前端地址	http://localhost:8080
coupon-service 本地验证地址	http://localhost:18081
inventory-service 本地验证地址	http://localhost:18082
Prometheus 本地地址	http://localhost:9090
Grafana 本地地址	http://localhost:3000

```
(base) PS C:\Users\user> docker --version
Docker version 29.4.0, build 9d7ad9f
PS C:\Users\user> kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1
PS C:\Users\user> minikube version
minikube version: v1.38.1
commit: c93a4cb9311efc66b98d33ea03f75f2c4120e9b0
PS C:\Users\user> helm version
version.BuildInfo{Version:"v4.1.4", GitCommit:"05fa37973dc9e42b76e1d2883494c87174b6074f", GitTreeState:"clean", GoVersion:"go1.25.9", KubeClientVersion:"v1.35"}
PS C:\Users\user> git --version
git version 2.53.0.windows.2
PS C:\Users\user> python --version
Python 3.12.8
```

图 2: Docker、kubectl、Minikube、Helm 等工具版本检查结果

2.3 仓库目录结构

仓库按照实验模块组织，便于每个成员独立提交、交叉检查与最终整合。核心目录如下：

```
1 online - boutique - testing - maintenance /
2 | -- README.md
3 | -- docs /
4 | -- scripts /
5 | -- deploy / online - boutique /
6 | -- services /
7 |   | -- coupon - service /
8 |   | -- inventory - service /
9 |   | -- integration /
10 | -- monitoring /
11 | -- chaos /
12 | -- tests /
```

```

13 |   |-- selenium/
14 |   |-- jmeter/
15 |-- data/
16 |-- algorithms/
17 |-- agent/
18 |-- figures/
19 |-- results/
20 |-- pdf/
21 |-- pptx/

```

Listing 2: 仓库核心目录结构

其中，services/ 保存新增微服务源码、Dockerfile 和 Kubernetes 配置；monitoring/ 保存 Prometheus、Grafana 配置和看板；chaos/ 保存 ChaosMesh 故障实验配置；tests/ 保存 Selenium 与 JMeter 测试材料；data/ 和 results/ 保存指标数据、统计表与分析结果；agent/ 保存智能运维 Agent 代码。

2.4 小组分工与个人任务

本项目由 6 名成员协作完成。为了避免工作重叠，小组按照模块边界分配任务，并要求每个模块同时交付代码、配置、命令、结果文件和截图。详细分工见表 4。

表 4: 小组成员分工与交付内容

成员	负责模块	具体工作	主要产出
王秀强	模块 A 队长、主系统部署、总集成、智能运维设计	确定 Online-Boutique 选型；建立并维护 GitHub 仓库；编写部署脚本；完成 Minikube 主系统部署；统一目录、命名、截图和结果文件；协调 B/C/D/E/F 模块的数据衔接；整理大作业报告主线和 PPT；设计并实现 AIOps Agent 的总体能力。汇总成员完成结果，协助制作答辩 PPT，负责上台答辩。	README、部署文档、命令记录、Pod 截图、Service 截图、前端页面截图、总体架构图、数据集验收报告、Agent 设计说明、报告与 PPT 统稿

成员	负责模块	具体工作	主要产出
邓锦尧	模块 B 新增微服务开发	设计优惠券与库存两个业务场景；开发 coupon-service 与 inventory-service；编写 FastAPI 接口、Prometheus 指标中间件、Dockerfile、requirements、Deployment、Service；配合完成 Minikube 镜像导入与接口验证。汇总成员完成结果，制作答辩 PPT	FastAPI 源码、Dockerfile、Kubernetes YAML、优惠券接口验证截图、库存接口验证截图、新增服务部署材料
邱俊杰	模块 C Prometheus、Grafana、监控看板、数据采集	部署 kube-prometheus-stack；配置 Prometheus Targets；接入 Online-Boutique、coupon-service 和 inventory-service 指标；设计 Grafana 看板；整理 PromQL 查询；参与 normal/fault/all 三类数据生成与质量校验。负责撰写实验报告中 Prometheus 部分内容。	Prometheus Targets 截图、Grafana 看板截图、PromQL 记录、指标说明表、normal/fault/all CSV、统计表
段坤良	模块 D ChaosMesh 故障实验、故障窗口标注	部署 ChaosMesh；编写并执行 Pod Kill、CPU 压力、内存压力、网络延迟、网络丢包等故障配置；记录故障对象、起止时间、系统现象和恢复情况；配合监控数据标注 fault_type 与 fault_phase。	ChaosMesh YAML、故障实验记录、故障前后 Grafana 截图、故障阶段标注、故障清理截图
韦厚林	模块 E Selenium 功能测试与 JMeter 性能测试	编写 Selenium 自动化脚本，覆盖首页访问、商品详情、加入购物车、购物车页面、结账操作；编写 JMeter 测试计划，覆盖 frontend、coupon、inventory 接口；执行 10/30/50/100 用户并发测试；整理响应时间、吞吐量、错误率等结果。负责撰写实验报告中 Selenium 功能测试部分。	Selenium 脚本、JMeter JMX、请求级结果 CSV、汇总结果 CSV、功能测试截图、性能测试图表

成员	负责模块	具体工作	主要产出
任泓旭	模块 F 论文算法复现与异常检测分析	基于 normal/fault/all 指标数据集选择 KPI 异常检测或故障诊断算法；围绕 Isolation Forest、PCA、One-Class SVM 等方法完成最小可运行实验；输出异常分数、检测结果和可视化图；配合报告撰写算法分析部分。	算法代码、异常检测图、论文复现说明、算法结果表

3 Online-Boutique 主系统部署

3.1 部署目标

主系统部署目标是把 Online-Boutique 完整运行在本地 Minikube 集群中，并能够通过浏览器访问前端页面。验收内容包括：Minikube 节点为 Ready，online-boutique 命名空间存在，核心 Pod 进入 Running，Service 正常创建，前端页面可访问，商品浏览、加入购物车和结账页面可用。

3.2 部署命令与执行说明

仓库提供了 PowerShell 脚本用于完成环境检查、旧资源清理、Minikube 启动、部署文件下载、镜像预加载、部署验证和前端端口转发。主要命令如下：

```
1 git clone https://github.com/Affordan/online-boutique-testing-maintenance
2 cd online-boutique-testing-maintenance
3
4 Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
5
6 .\scripts\check_env.ps1
7 .\scripts\clean_old_environment.ps1 -DeleteProjectProfile -StopOldProfile
8 .\scripts\run_fresh_deploy.ps1
9 .\scripts\wait_online_boutique.ps1
```

Listing 3: Online-Boutique 主系统部署命令

如果本地配置较低，可通过参数减少 Minikube 占用资源：


```
1 .\scripts\run_fresh_deploy.ps1 -Cpus 2 -Memory 4096
```

Listing 4: 低配置机器部署命令

若 Pod 出现 ErrImagePull 或 ImagePullBackOff, 说明节点拉取镜像失败, 此时不应重复部署, 而应预加载镜像并重启相关 Pod:

```
1 .\scripts\preload_images_to_minikube.ps1 -RestartPods
2 .\scripts\wait_online_boutique.ps1
```

Listing 5: 镜像拉取异常处理命令

前端访问通过端口转发完成:

```
1 Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
2 .\scripts\port_forward_frontend.ps1
```

Listing 6: 前端端口转发命令

随后浏览器访问 <http://localhost:8080>。当首页、商品详情页、购物车和结账页能够访问时, 说明主系统具备后续监控、故障和测试实验的基础。

3.3 部署结果验证

```
(base) PS C:\Users\user> kubectl config current-context
online-boutique-lab
(base) PS C:\Users\user> kubectl get nodes
NAME                STATUS    ROLES    AGE   VERSION
online-boutique-lab Ready    control-plane  12h   v1.35.1
(base) PS C:\Users\user> kubectl get pods -A
NAMESPACE   NAME                                                    READY   STATUS    RESTARTS   AGE
kube-system  coredns-7d764666f9-2h99p                               1/1     Running   0           12h
kube-system  etcd-online-boutique-lab                               1/1     Running   0           12h
kube-system  kube-apiserver-online-boutique-lab                     1/1     Running   0           12h
kube-system  kube-controller-manager-online-boutique-lab             1/1     Running   0           12h
kube-system  kube-proxy-nzpd4                                         1/1     Running   0           12h
kube-system  kube-scheduler-online-boutique-lab                      1/1     Running   0           12h
kube-system  storage-provisioner                                     1/1     Running   0           12h
online-boutique  adservice-7bb8787db5-nc29s                             1/1     Running   0           17m
online-boutique  cartservice-dc9f4fb6d-lvbvp                             1/1     Running   0           17m
online-boutique  checkoutservice-6d7ccb6cf-x9qd2                       1/1     Running   0           17m
online-boutique  currencyservice-7f9f695db5-8tv8m                      1/1     Running   0           17m
online-boutique  emailservice-58dc4788c5-qb95k                         1/1     Running   0           17m
online-boutique  frontend-76c94f7cfb-6ngx5                              1/1     Running   0           17m
online-boutique  loadgenerator-69fddd69d-vfhsr                          1/1     Running   0           17m
online-boutique  paymentservice-7cff85ccd7-mv2gc                       1/1     Running   0           17m
online-boutique  productcatalogservice-54c86d9f79-7rh44                 1/1     Running   0           17m
online-boutique  recommendationservice-6d8ddf5d5f-xtq2k                1/1     Running   0           17m
online-boutique  redis-cart-6899b65948-sclm8                            1/1     Running   0           17m
online-boutique  shippingservice-846f8585d7-hlxkv                      1/1     Running   0           17m
```

图 3: Minikube 节点状态验证结果

```
(base) PS E:\桌面文件\大学生文档文件\大三上\智能运维与测试\实验\大作业\第几小组-Online-Boutique> kubectl get pods -n online-boutique -w
```

NAME	READY	STATUS	RESTARTS	AGE
adservice-7bb8787db5-nc29s	0/1	Running	0	18s
cartservice-dc9f4fb6d-lvbvp	0/1	Running	0	18s
checkoutservice-6d7ccbb6cf-x9qd2	1/1	Running	0	18s
currencyservice-7f9f695db5-8tv8m	0/1	Running	0	18s
emailservice-58dc4788c5-qb95k	0/1	Running	0	18s
frontend-76c94f7cfb-6ngx5	0/1	Running	0	18s
loadgenerator-69fddd69d-vfhsr	0/1	Init:0/1	0	18s
paymentservice-7cff85ccd7-mv2gc	0/1	Running	0	18s
productcatalogservice-54c86d9f79-7rh44	1/1	Running	0	17s
recommendationservice-6d8ddf5d5f-xtq2k	0/1	Running	0	17s
redis-cart-6899b65948-sclm8	0/1	Running	0	17s
shippingservice-846f8585d7-hlxkv	1/1	Running	0	17s
loadgenerator-69fddd69d-vfhsr	0/1	Init:ErrImagePull	0	22s
emailservice-58dc4788c5-qb95k	1/1	Running	0	23s
recommendationservice-6d8ddf5d5f-xtq2k	1/1	Running	0	22s
redis-cart-6899b65948-sclm8	1/1	Running	0	23s
frontend-76c94f7cfb-6ngx5	1/1	Running	0	28s
paymentservice-7cff85ccd7-mv2gc	1/1	Running	0	28s
currencyservice-7f9f695db5-8tv8m	1/1	Running	0	29s
loadgenerator-69fddd69d-vfhsr	0/1	Init:ImagePullBackOff	0	34s
cartservice-dc9f4fb6d-lvbvp	1/1	Running	0	38s
loadgenerator-69fddd69d-vfhsr	0/1	Init:0/1	0	38s
adservice-7bb8787db5-nc29s	1/1	Running	0	48s
loadgenerator-69fddd69d-vfhsr	0/1	PodInitializing	0	53s
loadgenerator-69fddd69d-vfhsr	1/1	Running	0	54s

图 4: Online-Boutique 核心 Pod 运行状态

```
(base) PS E:\桌面文件\大学生文档文件\大三上\智能运维与测试\实验\大作业\第几小组-Online-Boutique> kubectl get svc -n online-boutique
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
adservice	ClusterIP	10.105.57.22	<none>	9555/TCP	11h
cartservice	ClusterIP	10.96.101.77	<none>	7070/TCP	11h
checkoutservice	ClusterIP	10.109.157.7	<none>	5050/TCP	11h
currencyservice	ClusterIP	10.110.183.153	<none>	7000/TCP	11h
emailservice	ClusterIP	10.100.76.14	<none>	5000/TCP	11h
frontend	ClusterIP	10.106.8.237	<none>	80/TCP	11h
frontend-external	LoadBalancer	10.98.92.180	<pending>	80:32714/TCP	11h
paymentservice	ClusterIP	10.105.14.195	<none>	50051/TCP	11h
productcatalogservice	ClusterIP	10.102.147.18	<none>	3550/TCP	11h
recommendationservice	ClusterIP	10.111.47.72	<none>	8080/TCP	11h
redis-cart	ClusterIP	10.100.246.123	<none>	6379/TCP	11h
shippingservice	ClusterIP	10.98.45.186	<none>	50051/TCP	11h

图 5: Online-Boutique Service 列表

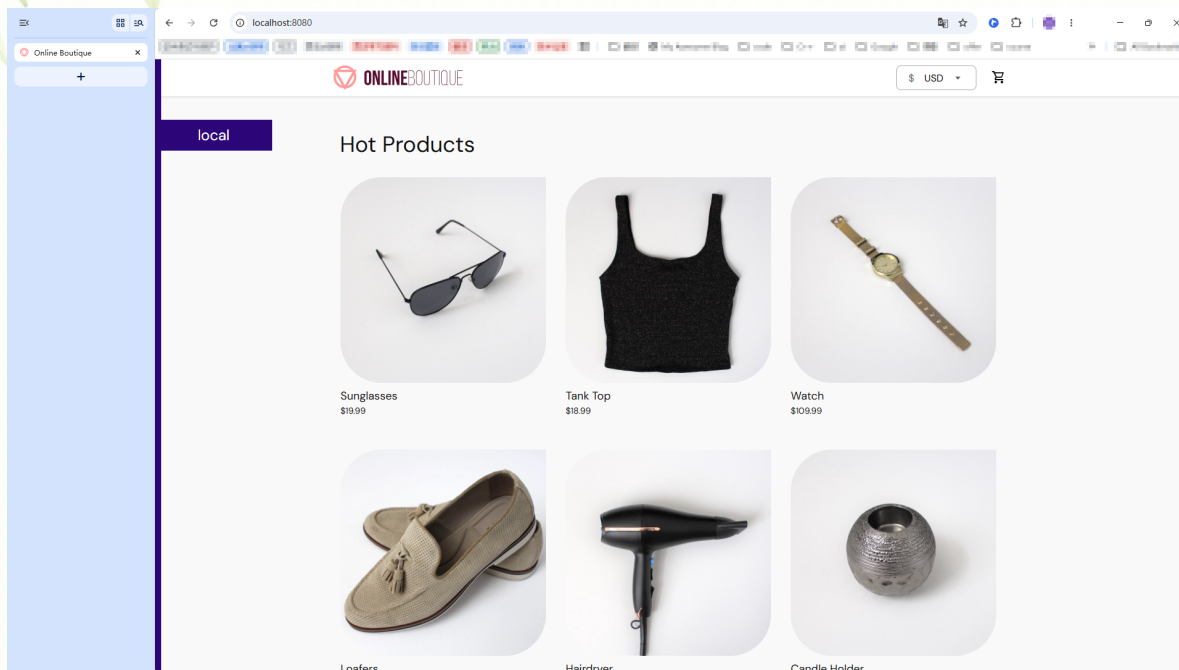


图 6: Online-Boutique 前端首页访问结果

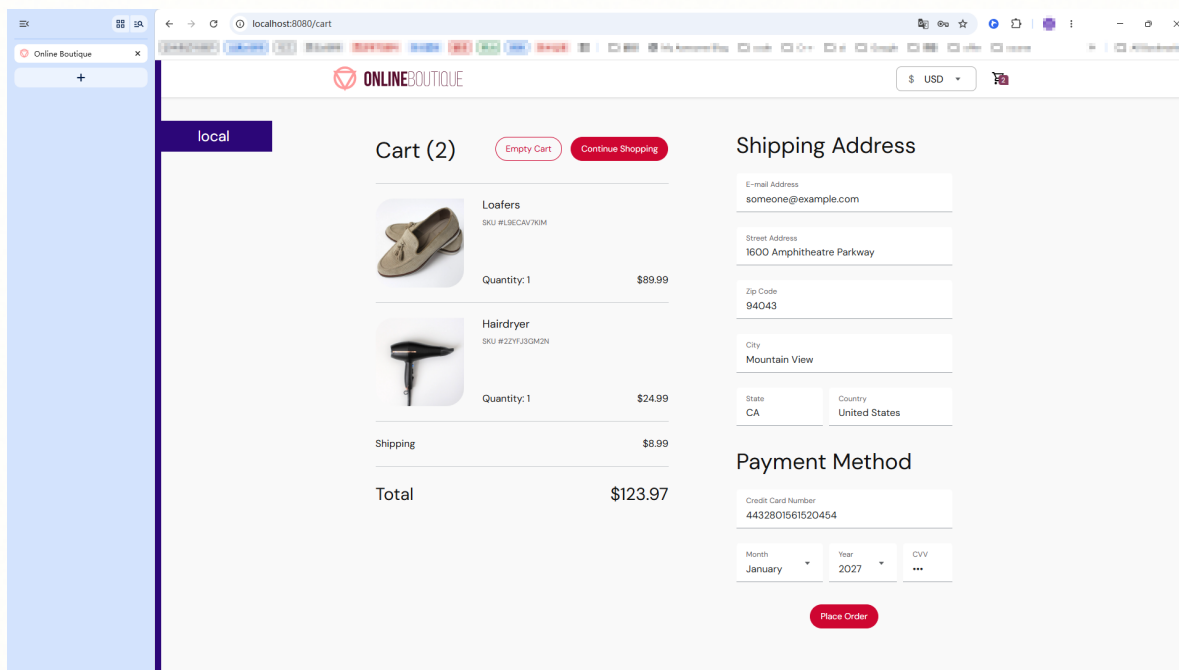


图 7: 购物车页面访问结果

从以上截图可以看到,Minikube 节点已经就绪,Online-Boutique 微服务 Pod 处于 Running 状态,前端服务可通过本地端口访问。主系统部署完成后,后续新增微服务、Prometheus 监控、ChaosMesh 故障实验和测试任务均以该环境为基础。

4 新增微服务开发与接入

4.1 新增服务设计目标

为了满足第三档要求，项目在 Online-Boutique 原系统上新增两个业务微服务。新增服务选择优惠券和库存两个场景，原因是它们与电商核心链路关系紧密，能够自然接入前端、商品目录和结账服务，同时也适合暴露应用级监控指标。

表 5: 新增微服务说明

服务	作用	接入对象	集群内地址
coupon-service	优惠券与促销服务	frontend、checkoutservice	http://coupon-service:8080
inventory-service	商品库存服务	productcatalogservice、checkoutservice	http://inventory-service:8080

当前原 Online-Boutique 前端页面没有直接改造优惠券输入框或库存数量展示，因此新增服务主要通过 REST 接口、Kubernetes Service、环境变量注入和 API 压测完成验证。这样能在不重新构建官方前端镜像的前提下完成服务开发与接入，减少团队协作中的镜像分发成本。

4.2 coupon-service 实现

coupon-service 负责优惠券列表查询、优惠码校验、结账优惠应用和订单优惠记录查询。服务内部预置 SAVE10、WELCOME5 和 VIP15 三类促销规则，并根据订单金额计算折扣。主要接口见表 6。

表 6: coupon-service 接口

接口	方法	说明
/health	GET	健康检查
/coupons	GET	查询可用优惠券
/coupons/validate	POST	校验优惠码并计算折扣
/coupons/apply	POST	结账时应用优惠
/coupons/applied/{order_id}	GET	查询订单优惠记录
/metrics	GET	Prometheus 指标输出

服务通过 Prometheus Counter 统计请求次数，通过 Histogram 统计请求耗时。指标名包括 coupon_service_requests_total 和 coupon_service_request_duration_seconds，标签包含

service、method、endpoint、status。这样 Grafana 能够展示优惠券服务的请求速率、错误率和延迟。

```
1 REQUEST_COUNT = Counter(  
2     "coupon_service_requests_total",  
3     "HTTP requests",  
4     ["service", "method", "endpoint", "status"]  
5 )  
6 REQUEST_LATENCY = Histogram(  
7     "coupon_service_request_duration_seconds",  
8     "HTTP latency",  
9     ["service", "method", "endpoint", "status"]  
10 )
```

Listing 7: coupon-service 指标中间件核心逻辑

4.3 inventory-service 实现

inventory-service 负责库存列表查询、单个商品库存查询、库存同步、库存预留和库存释放。服务内部维护商品库存与已预留数量，计算可用库存，并在库存不足时返回 409 错误。主要接口见表 7。

表 7: inventory-service 接口

接口	方法	说明
/health	GET	健康检查
/inventory	GET	查询库存列表
/inventory/{product_id}	GET	查询指定商品库存
/inventory/sync	POST	同步商品库存
/inventory/reserve	POST	下单时预留库存
/inventory/release	POST	取消订单时释放库存
/metrics	GET	Prometheus 指标输出

库存服务同样暴露 Prometheus 指标，指标名包括 inventory_service_requests_total 和 inventory_service_request_duration_seconds。这些指标补齐了原系统中应用级请求量与延迟难以直接观察的问题。

4.4 容器构建、部署与环境变量注入

新增服务部署由脚本 `deploy_custom_services.ps1` 完成。该脚本构建 `coupon-service` 和 `inventory-service` 镜像，将镜像导入 Minikube，部署 Kubernetes 资源，并向原系统相关 Deployment 注入新增服务地址。

```
1 .\scripts\deploy_custom_services.ps1
2 .\scripts\verify_custom_services.ps1
3
4 .\scripts\port_forward_coupon.ps1
5 .\scripts\test_coupon_service.ps1
6
7 .\scripts\port_forward_inventory.ps1
8 .\scripts\test_inventory_service.ps1
```

Listing 8: 新增微服务部署与验证命令

环境变量注入关系见表 8。

表 8: 新增服务地址注入关系

Deployment	注入环境变量
frontend	COUPON_SERVICE_ADDR=http://coupon-service:8080; INVENTORY_SERVICE_ADDR=http://inventory-service:8080
checkoutservice	COUPON_SERVICE_ADDR=http://coupon-service:8080; INVENTORY_SERVICE_ADDR=http://inventory-service:8080
productcatalogservice	INVENTORY_SERVICE_ADDR=http://inventory-service:8080

4.5 新增服务验证结果

```
(base) PS C:\Users\jlfan> kubectl get pods -n online-boutique | findstr "coupon inventory"
coupon-service-7d4496885c-ht742      1/1      Running    0      42m
inventory-service-58849fb677-zchzj    1/1      Running    0      42m
```

图 8: 新增微服务 Pod 运行状态

```
(base) PS C:\Users\jlfan> kubectl get svc -n online-boutique | findstr "coupon inventory"
coupon-service      ClusterIP      10.102.184.198    <none>      8080/TCP      43m
inventory-service   ClusterIP      10.107.103.189    <none>      8080/TCP      43m
```

图 9: 新增微服务 Service 状态


```
(base) PS C:\Users\Affordan> Invoke-RestMethod http://localhost:18081/health

status service           integrated_with
-----
ok      coupon-service frontend, checkoutservice

(base) PS C:\Users\Affordan> Invoke-RestMethod http://localhost:18081/coupons

promotions
-----
{@{code=SAVE10; type=percent; value=10; min_amount=20; description=10% off orders over $20}, @{{code=WELCOME5; type=fix...

(base) PS C:\Users\Affordan> Invoke-RestMethod `
>> -Method Post `
>> -Uri http://localhost:18081/coupons/validate `
>> -ContentType "application/json" `
>> -Body '{"code":"SAVE10","cart_total":49.99}'

code      : SAVE10
valid     : True
discount  : 5
final_total : 44.99
promotion : @{{code=SAVE10; type=percent; value=10; min_amount=20; description=10% off orders over $20}
caller    : frontend
checked_at : 2026/6/3 19:42:57
```

图 10: 优惠券服务接口验证结果

```
(base) PS C:\Users\Affordan> Invoke-RestMethod http://localhost:18082/health

status service           integrated_with
-----
ok      inventory-service productcatalogservice, checkoutservice

(base) PS C:\Users\Affordan> Invoke-RestMethod http://localhost:18082/inventory

items
-----
{@{product_id=OLJCESPC7Z; name=Vintage Typewriter; stock=25; reserved=0; available=25; catalog_addr=productcatalogserv...

(base) PS C:\Users\Affordan> Invoke-RestMethod http://localhost:18082/inventory/OLJCESPC7Z

product_id : OLJCESPC7Z
name       : Vintage Typewriter
stock      : 25
reserved   : 0
available  : 25
caller     : productcatalogservice
checked_at : 2026/6/3 19:43:43
```

图 11: 库存服务接口验证结果

```
(base) PS C:\Users\Affordan> .\scripts\verify_custom_services.ps1
===> Custom service pods
coupon-service-7d4496885c-ht742      1/1      Running   0          48m
inventory-service-58849fb677-zchzj    1/1      Running   0          48m
===> Custom service services
coupon-service      ClusterIP   10.102.184.198 <none>     8080/TCP    48m
inventory-service   ClusterIP   10.107.103.189 <none>     8080/TCP    48m
===> frontend env
COUPON_SERVICE_ADDR: http://coupon-service:8080
INVENTORY_SERVICE_ADDR: http://inventory-service:8080
===> checkoutservice env
COUPON_SERVICE_ADDR: http://coupon-service:8080
INVENTORY_SERVICE_ADDR: http://inventory-service:8080
===> productcatalogservice env
INVENTORY_SERVICE_ADDR: http://inventory-service:8080
===> Test commands
kubectl port-forward svc/coupon-service 18081:8080 -n online-boutique
kubectl port-forward svc/inventory-service 18082:8080 -n online-boutique
Invoke-RestMethod http://localhost:18081/health
Invoke-RestMethod http://localhost:18081/coupons
Invoke-RestMethod http://localhost:18082/health
Invoke-RestMethod http://localhost:18082/inventory/OLJCESPC7Z
```

图 12: 新增服务地址注入原系统验证结果

从验证结果看, coupon-service 和 inventory-service 均已在 online-boutique namespace 中运行, Service 类型为 ClusterIP, 接口验证能够正常返回数据, 原系统相关 Deployment 也完

成新增服务地址注入。

5 Prometheus 与 Grafana 监控实验

5.1 监控目标

监控实验目标是采集 Online-Boutique 与新增微服务运行指标，并为故障注入、性能测试、异常数据整理和 Agent 诊断提供数据基础。监控对象包括 Kubernetes 层面的 Pod 状态、重启次数、CPU、内存、网络流量，也包括新增服务暴露的请求量、错误率、平均延迟和 P95 延迟。

5.2 监控组件部署

项目使用 Helm 安装 kube-prometheus-stack，其中包含 Prometheus、Grafana、kube-state-metrics、node-exporter 和 Alertmanager。部署命令如下：

```
1 .\scripts\deploy_monitoring.ps1
2 .\scripts\port_forward_prometheus.ps1
3 .\scripts\port_forward_grafana.ps1
```

Listing 9: Prometheus 与 Grafana 部署命令

Prometheus 访问地址为 `http://localhost:9090`，Grafana 访问地址为 `http://localhost:3000`。Grafana 默认账号为 admin，密码通过 Kubernetes Secret 查询：

```
1 kubectl get secret -n monitoring monitoring-stack-grafana -o jsonpath='{.data.admin-password}' | base64 --decode
```

Listing 10: Grafana 密码查询命令

5.3 指标接入设计

Online-Boutique 原始服务即使没有完整应用级 `/metrics`，也能通过 kubelet、cAdvisor 和 kube-state-metrics 观察容器资源指标。新增服务提供应用级指标端点，Service 添加 `monitoring: enabled` 标签，由 ServiceMonitor 自动发现。

```
1 metadata:
2   labels:
3     monitoring: enabled
```



```
4 ports:
5   - name: http
6     port: 8080
7     targetPort: 8080
```

Listing 11: 新增服务 ServiceMonitor 采集约定

常用 PromQL 如下:

```
1 sum by (phase) (kube_pod_status_phase{namespace="online - boutique"})
2
3 sum by (pod) (
4   rate(container_cpu_usage_seconds_total{namespace="online - boutique"}[5m
5     ])
6
7 sum by (pod) (
8   container_memory_working_set_bytes{namespace="online - boutique"}
9 )
10
11 sum by (service) (
12   rate({__name__=~"coupon_service_requests_total|
13     inventory_service_requests_total",
14     namespace="online - boutique"}[5m])
15
16 histogram_quantile(0.95,
17   sum by (le, service) (
18     rate({__name__=~"coupon_service_request_duration_seconds_bucket|
19     inventory_service_request_duration_seconds_bucket",
20     namespace="online - boutique"}[5m])
21 )
```

Listing 12: PromQL 查询示例

5.4 监控结果展示

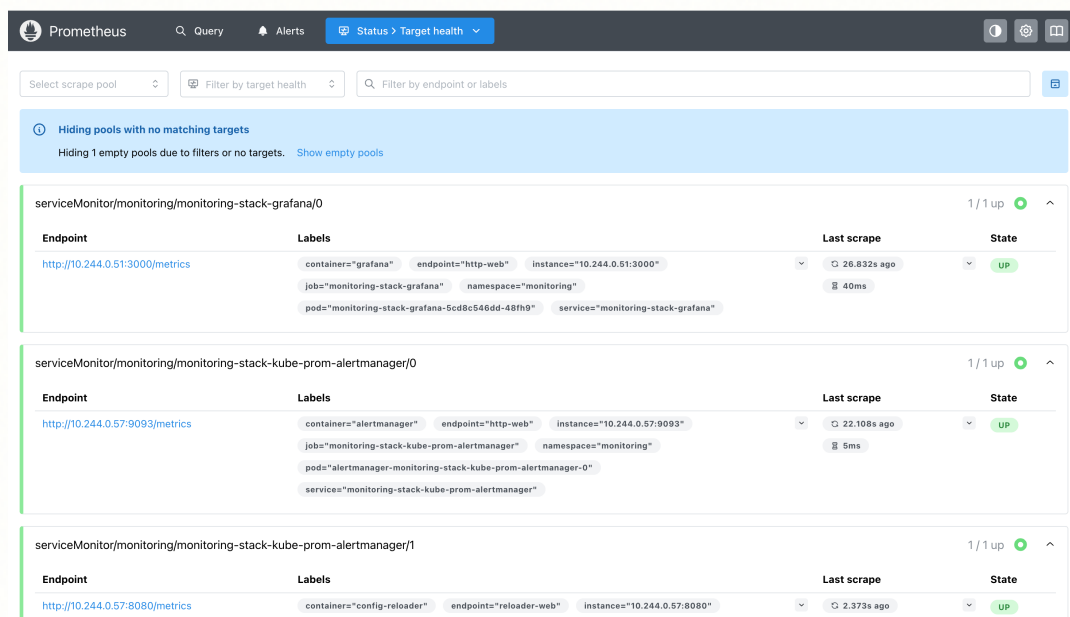


图 13: Prometheus Targets 采集状态

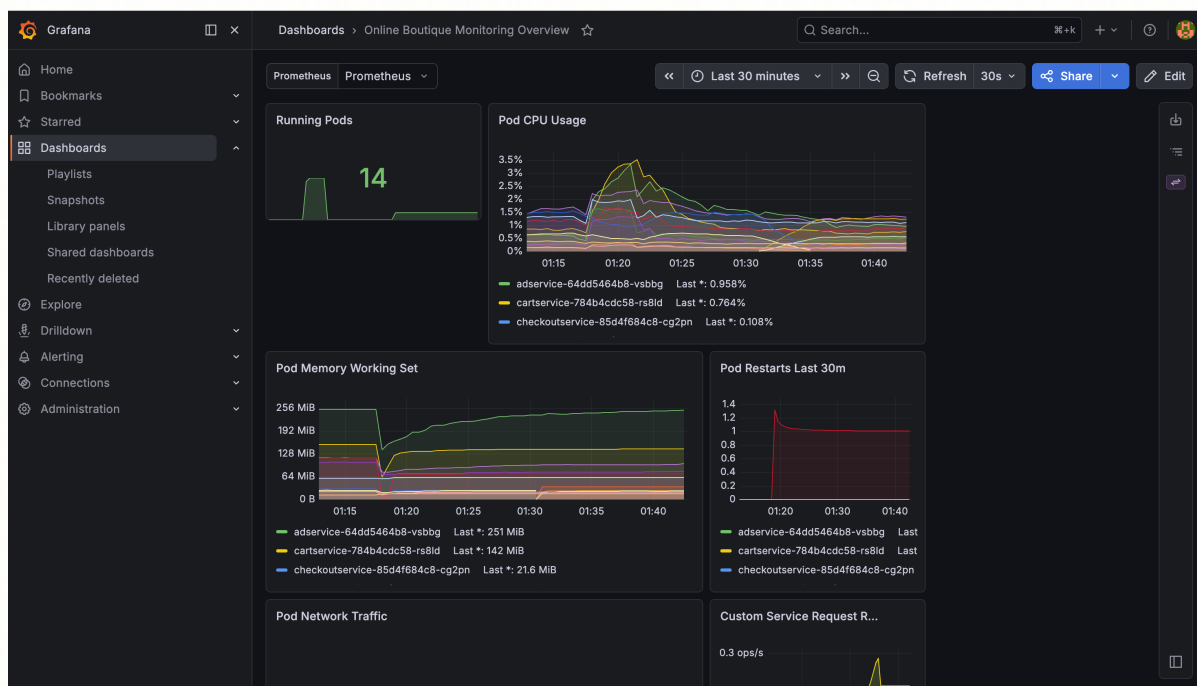


图 14: Online-Boutique Grafana 总览看板

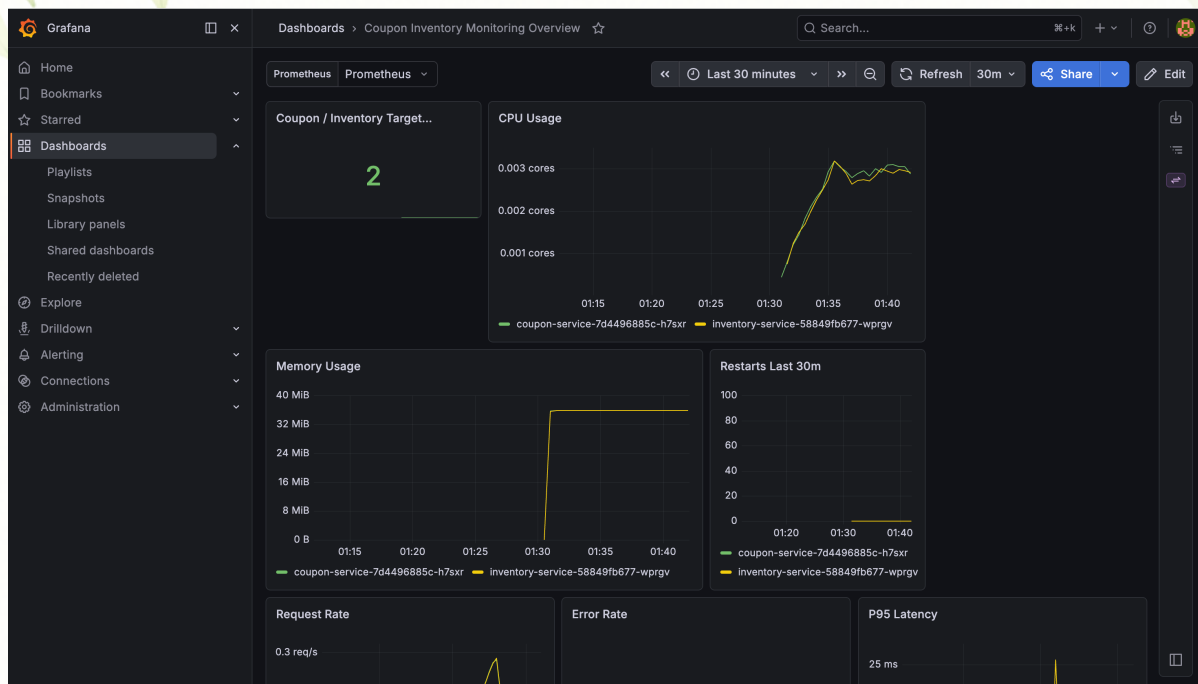


图 15: 新增优惠券和库存服务监控面板

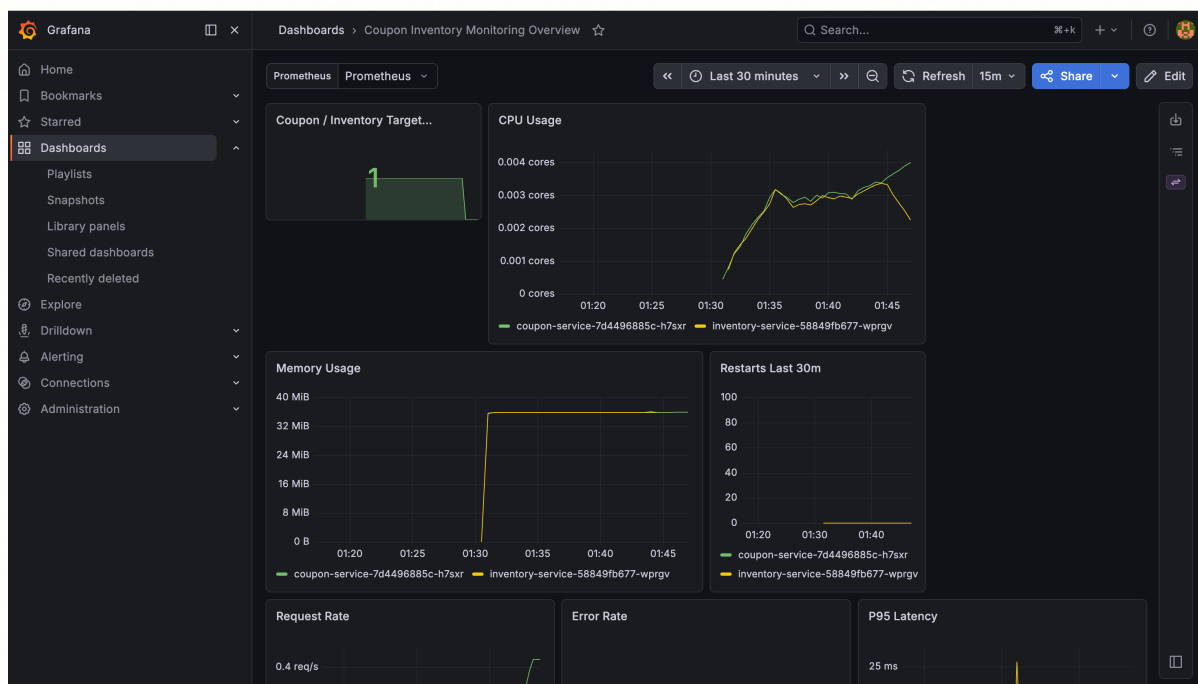


图 16: 故障前后监控指标对比面板

Prometheus Targets 页面用于验证采集端点是否 UP。Grafana 总览看板用于观察整体资源状态；新增服务面板用于展示 coupon-service 与 inventory-service 的请求量、延迟和错误率；故障对比面板用于支持后续 ChaosMesh 实验分析。

6 ChaosMesh 故障注入实验

6.1 故障注入目标

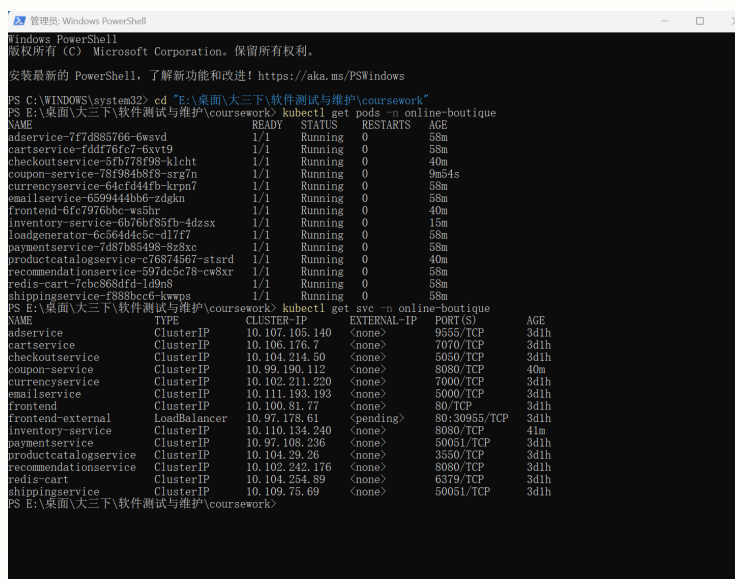
故障注入实验用于观察微服务系统在不同异常场景下的行为，并为异常数据集提供带标签的故障窗口。本项目使用 ChaosMesh 注入 Pod Kill、CPU 压力、内存压力、网络延迟、网络丢包等故障。每次实验记录故障名称、目标服务、开始时间、结束时间、故障配置、页面现象、Grafana 指标变化和系统恢复情况。

6.2 ChaosMesh 安装与验证

ChaosMesh 使用 Helm 安装。安装后需确认 chaos-mesh 命名空间下的组件 Pod 正常运行。

```
1 helm repo add chaos-mesh https://charts.chaos-mesh.org
2 helm repo update
3 kubectl create ns chaos-mesh
4 helm install chaos-mesh chaos-mesh/chaos-mesh -n chaos-mesh --version 2.7.0
5 kubectl get pods -n chaos-mesh
```

Listing 13: ChaosMesh 安装与验证命令



```
Windows PowerShell
版权所有 (C) Microsoft Corporation. 保留所有权利。

安装最新的 PowerShell，了解新功能和改进！https://aka.ms/PSWindows

PS C:\WINDOWS\system32> cd "E:\桌面\大三下\软件测试与维护\coursework"
PS E:\桌面\大三下\软件测试与维护\coursework> kubectl get pods -n online-boutique
NAME          READY   STATUS    RESTARTS   AGE
adservice-7f7d885766-gwsvd    1/1     Running   0          58m
cartservice-fdd4f6fc7-6xvt9   1/1     Running   0          58m
checkoutservice-3f5778f98-kicht 1/1     Running   0          40m
coupon-service-78f984b3f8-srzfn 1/1     Running   0          9m54s
currencyservice-64cf044fb-krpn7 1/1     Running   0          58m
emailservice-6599444bb6-zdgkn 1/1     Running   0          58m
frontend-6fc7976b6c-ws5hr     1/1     Running   0          40m
inventoryservice-6b76b685fb-4dzsx 1/1     Running   0          15m
loadgenerator-6c564d4c5c-dl7f7 1/1     Running   0          58m
paymentservice-7d87b85498-8z8xc 1/1     Running   0          58m
productcatalogservice-c76874567-stsrd 1/1     Running   0          40m
recommendationservice-297dc5c78-cw8ur 1/1     Running   0          58m
redis-cart-7cbc8684fd-ld9n8    1/1     Running   0          58m
shippingservice-f888bccc6-kwpps 1/1     Running   0          58m
PS E:\桌面\大三下\软件测试与维护\coursework> kubectl get svc -n online-boutique
NAME          TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)          AGE
adservice     ClusterIP   10.107.105.140   <none>        9555/TCP         3d1h
cartservice   ClusterIP   10.106.176.7     <none>        7070/TCP         3d1h
checkoutservice ClusterIP   10.104.214.50    <none>        5050/TCP         3d1h
coupon-service ClusterIP   10.99.190.112    <none>        8080/TCP         40s
currencyservice ClusterIP   10.102.211.220   <none>        7000/TCP         3d1h
emailservice   ClusterIP   10.111.193.193   <none>        5000/TCP         3d1h
frontend      ClusterIP   10.100.81.77     <none>        80/TCP           3d1h
frontend-external LoadBalancer 10.97.178.61     <pending>    80:30955/TCP    3d1h
inventoryservice ClusterIP   10.110.134.240   <none>        8080/TCP         41s
paymentservice ClusterIP   10.97.108.236    <none>        50051/TCP        3d1h
productcatalogservice ClusterIP   10.104.29.26     <none>        3550/TCP         3d1h
recommendationservice ClusterIP   10.102.242.176   <none>        8080/TCP         3d1h
redis-cart     ClusterIP   10.104.254.89    <none>        6379/TCP         3d1h
shippingservice ClusterIP   10.109.75.69     <none>        50051/TCP        3d1h
PS E:\桌面\大三下\软件测试与维护\coursework>
```

图 17: 故障注入前 Online-Boutique Pod 初始状态


```

shippingservice      ClusterIP      10.109.75.69      <none>      50051/TCP      3d1h
PS E:\桌面\大三下\软件测试与维护\coursework> kubectl get pods -n chaos-mesh
NAME                                READY    STATUS    RESTARTS      AGE
chaos-controller-manager-5f5f85ff55-44cs8    1/1      Running   1 (64m ago)    3d1h
chaos-controller-manager-5f5f85ff55-6z6c6    1/1      Running   1 (64m ago)    3d1h
chaos-controller-manager-5f5f85ff55-sgc8d    1/1      Running   1 (64m ago)    3d1h
chaos-daemon-9fbcc                          1/1      Running   1 (64m ago)    3d1h
chaos-dashboard-79cbbd84b-7xjbm             1/1      Running   1 (64m ago)    3d1h
chaos-dns-server-68477f5bc5-cl76k          1/1      Running   1 (64m ago)    3d1h
PS E:\桌面\大三下\软件测试与维护\coursework>

```

图 18: ChaosMesh 组件 Pod 运行状态

6.3 故障实验设计

本项目设计了 8 组故障实验，覆盖新增服务和原有核心服务。实验列表见表 9。

表 9: ChaosMesh 故障实验列表

编号	故障类型	目标服务	配置文件或结果说明
F001	Pod Kill	coupon-service	chaos_pod_kill_coupon.yaml
F002	Pod Kill	inventory-service	chaos_pod_kill_inventory.yaml
F003	CPU Stress	coupon-service	chaos_cpu_stress_coupon.yaml
F004	Network Delay	inventory-service	chaos_network_delay_inventory.yaml
F005	Network Delay	coupon-service	chaos_network_delay_coupon.yaml
F006	CPU Stress	inventory-service	已完成
F007	Network Loss	coupon-service	已完成
F008	Network Loss	inventory-service	已完成

通用执行命令如下：

```

1 kubectl apply -f chaos/<config-file>.yaml
2 kubectl get podchaos, stresschaos, networkchaos -n chaos-mesh
3 kubectl describe <chaos-type> <experiment-name> -n chaos-mesh
4 kubectl delete <chaos-type> <experiment-name> -n chaos-mesh

```

Listing 14: ChaosMesh 故障注入通用命令

6.4 代表性实验结果

```
chaos-dns-server-6847f15bc5-c176k 1/1 Running 1 (64m ago) 3d1h
PS E:\桌面\大三下\软件测试与维护\coursework> type .\chaos\chaos_pod_kill_coupon.yaml
apiVersion: chaos-mesh.org/v1alpha1
kind: PodChaos
metadata:
  name: pod-kill-coupon-f001
  namespace: chaos-mesh
spec:
  action: pod-kill
  mode: one
  duration: 10m
  selector:
    namespaces:
      - online-boutique
    labelSelectors:
      app: coupon-service
PS E:\桌面\大三下\软件测试与维护\coursework>
```

图 19: F001 Pod Kill 实验 YAML 配置

```
PS E:\桌面\大三下\软件测试与维护\coursework> kubectl apply -f .\chaos\chaos_pod_kill_coupon.yaml
podchaos.chaos-mesh.org/pod-kill-coupon-f001 unchanged
PS E:\桌面\大三下\软件测试与维护\coursework>
```

图 20: F001 Pod Kill 实验创建成功

```
PS E:\桌面\大三下\软件测试与维护\coursework> kubectl describe podchaos pod-kill-coupon-f001 -n chaos-mesh
Name:         pod-kill-coupon-f001
Namespace:    chaos-mesh
Labels:       <none>
Annotations:  <none>
API Version:  chaos-mesh.org/v1alpha1
Kind:         PodChaos
Metadata:
  Creation Timestamp: 2026-06-06T12:48:39Z
  Finalizers:
    chaos-mesh/records
  Generation: 5
  Resource Version: 56133
  UID: 0d6f23f-94d1-4973-bac5-5d6f9b52aff
Spec:
  Action: pod-kill
  Duration: 10m
  Mode: one
  Selector:
    Label Selectors:
      App: coupon-service
    Namespaces:
      online-boutique
Status:
  Conditions:
    Status: True
    Type: Selected
    Status: True
    Type: AllInjected
    Status: False
    Type: AllRecovered
    Status: False
    Type: Paused
  Experiment:
  Container Records:
    Events:
      Operation: Apply
      Timestamp: 2026-06-06T12:48:40Z
      Type: Succeeded
      Id: online-boutique/coupon-service-78f984b8f8-flkhv
      Injected Count: 1
      Phase: Injected
      Recovered Count: 0
      Selector Key:
      Desired Phase: Run
  Events:
    Type Reason Age From Message
    ----
    Normal FinalizerInitiated 26m initFinalizers Finalizer has been initiated
    Normal Updated 26m initFinalizers Successfully update finalizer of resource
    Normal Updated 26m desiredPhase Successfully update desiredPhase of resource
    Normal Applied 26m records Successfully apply chaos for online-boutique/coupon-service-78f984b8f8-flkhv
    Normal Updated 26m records Successfully update records of resource
PS E:\桌面\大三下\软件测试与维护\coursework>
```

图 21: F001 Pod Kill 实验详情


```

PS E:\桌面\大三下\软件测试与维护\coursework> kubectl apply -f chaos/chaos_cpu_stress_coupon.yaml
stresschaos.chaos-mesh.org/cpu-stress-coupon-f003 created
PS E:\桌面\大三下\软件测试与维护\coursework> kubectl get stresschaos -n chaos-mesh
NAME                                DURATION
cpu-stress-coupon-f003             15m
PS E:\桌面\大三下\软件测试与维护\coursework>

```

图 22: F003 CPU Stress 实验创建成功

```

PS E:\桌面\大三下\软件测试与维护\coursework> kubectl describe networkchaos network-delay-inventory-f004 -n chaos-mesh
Name: network-delay-inventory-f004
Namespace: chaos-mesh
Labels: <none>
Annotations: <none>
API Version: chaos-mesh.org/v1alpha1
Kind: NetworkChaos
Metadata:
  CreationTimestamp: 2026-06-06T15:24:27Z
  Finalizers:
    - chaos-mesh/records
  Generation: 1
  Resource Version: 66407
  UID: cdf086a2-7a94-4511-9344-20748c716538
Spec:
  Action: delay
  Delay:
    Correlation: 50
    Jitter: 1000ms
    Latency: 3000ms
    Direction: to
    Duration: 15m
    Mode: one
  Selector:
    Label Selectors:
      App: inventory-service
      Namespace: online-boutique
Status:
  Conditions:
    Status: True
    Type: Selected
    Status: True
    Type: AllInjected
    Status: False
    Type: AllRecovered
    Status: False
    Type: Paused
  Experiment:
    Container Records:
      Events:
        Operation: Apply
        Timestamp: 2026-06-06T15:24:28Z
        Type: Succeeded
        Id: online-boutique/inventory-service-6b76bf85fb-w91s6
        Injected Count: 1
        Phase: Injected
        Recovered Count: 0
        Selector Key:
        Desired Phase: Run
      Instances:
        online-boutique/inventory-service-6b76bf85fb-w91s6: 1
    Events:
      Type Reason Age From Message
      ----
      Normal FinalizerInitiated 2m14s initFinalizers Finalizer has been initiated
      Normal Updated 2m14s initFinalizers Successfully update finalizer of resource
      Normal Started 2m14s desiredPhase Experiment has started
      Normal Updated 2m14s desiredPhase Successfully update desiredPhase of resource
      Normal Updated 2m13s records Successfully update records of resource
      Normal Applied 2m13s records Successfully apply chaos for online-boutique/inventory-service-6b76bf85fb-w91s6
      Normal Updated 2m13s records Successfully update records of resource
PS E:\桌面\大三下\软件测试与维护\coursework>

```

图 23: F004 Network Delay 实验详情

```

networkchaos.chaos-mesh.org/network-loss-inventory-f008 created
PS E:\桌面\大三下\软件测试与维护\coursework> kubectl delete networkchaos network-loss-coupon-f007 -n chaos-mesh
networkchaos.chaos-mesh.org "network-loss-coupon-f007" deleted from chaos-mesh namespace
PS E:\桌面\大三下\软件测试与维护\coursework>

```

图 24: F007 Network Loss 实验详情

```

Normal Updated 2m38s records Successfully update records of resource
PS E:\桌面\大三下\软件测试与维护\coursework> kubectl delete networkchaos network-loss-inventory-f008 -n chaos-mesh
networkchaos.chaos-mesh.org "network-loss-inventory-f008" deleted from chaos-mesh namespace
PS E:\桌面\大三下\软件测试与维护\coursework>

```

图 25: F008 Network Loss 实验清理完成

故障注入实验不仅用于演示系统异常，还用于生成 fault_metrics.csv 中的故障标签。Pod Kill 主要引起重启次数变化与短时请求失败；CPU Stress 主要引起 CPU 使用率和延迟升高；Network Delay 与 Network Loss 主要影响 P95/P99 延迟、错误率和吞吐量。这些现象在监控数据中形成明显波动，为后续算法分析和 Agent 诊断提供依据。

7 Selenium 功能测试与 JMeter 性能测试

7.1 测试目标

测试模块承担两个任务：一是用 Selenium 验证 Online-Boutique 前端主业务链是否可用；二是用 JMeter 对前端、新增优惠券服务和新增库存服务进行并发请求测试，生成性能数据并驱动监控指标变化。

由于原 Online-Boutique 前端没有直接展示优惠券输入框和库存数量，新增服务测试采用“API 保底，前端增强”的策略：Selenium 覆盖原系统主业务链，JMeter 直接压测新增服务接口。这样既能证明原系统业务可访问，也能证明新增服务具备独立服务能力和压测价值。

7.2 Selenium 功能测试

Selenium 测试覆盖以下用户操作：打开首页、进入商品详情、加入购物车、打开购物车、填写结账信息、提交订单。主链路如下：

- 1 打开首页 -> 进入商品详情 -> 加入购物车 -> 打开购物车 -> 填写结账信息 -> 提交订单

Listing 15: Selenium 主业务链

执行脚本位于 tests/selenium/selenium_test.py, 结果记录保存至 results/testing/selenium_result.md

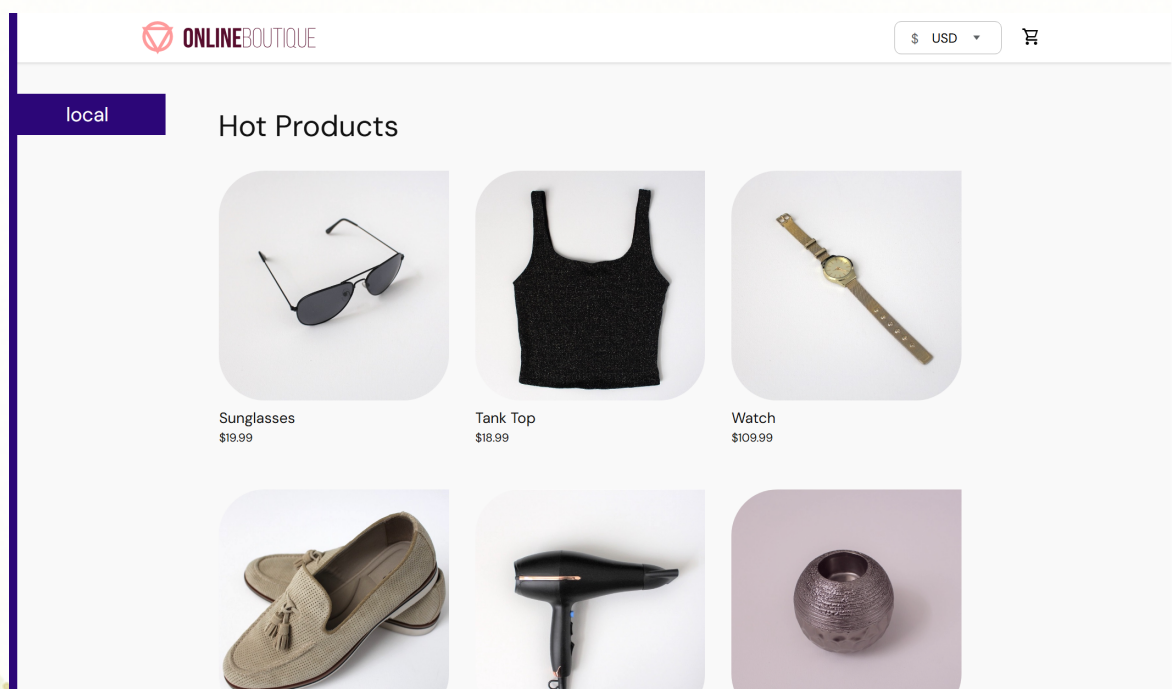


图 26: Selenium 打开 Online-Boutique 首页

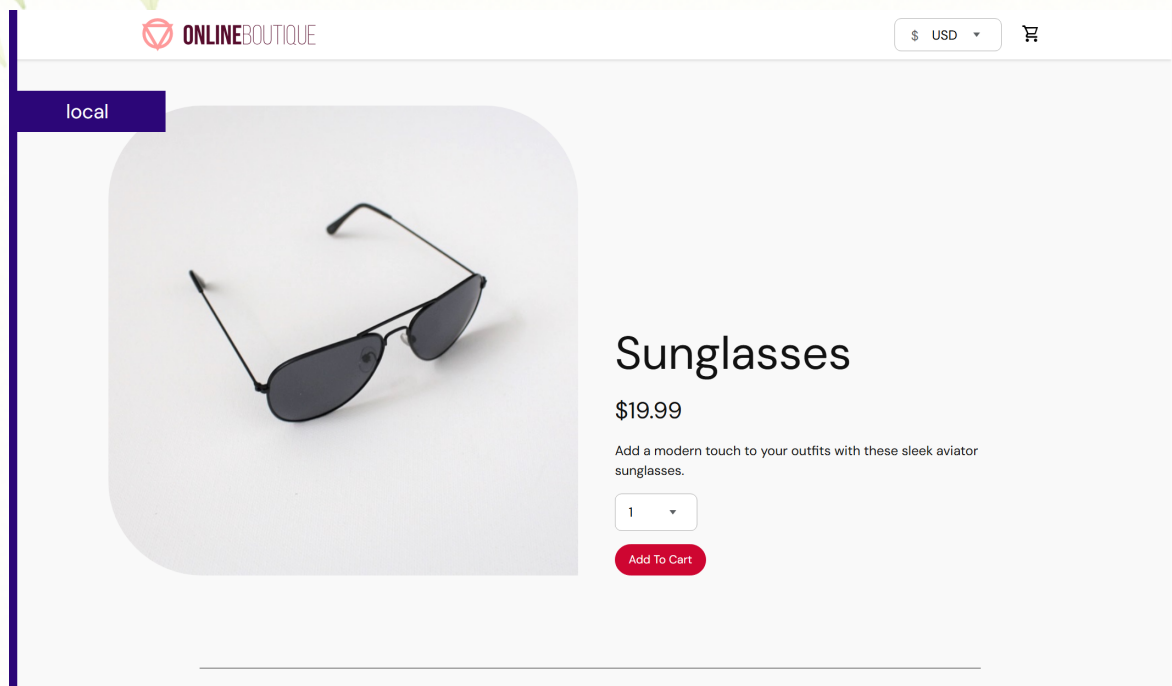


图 27: Selenium 进入商品详情页

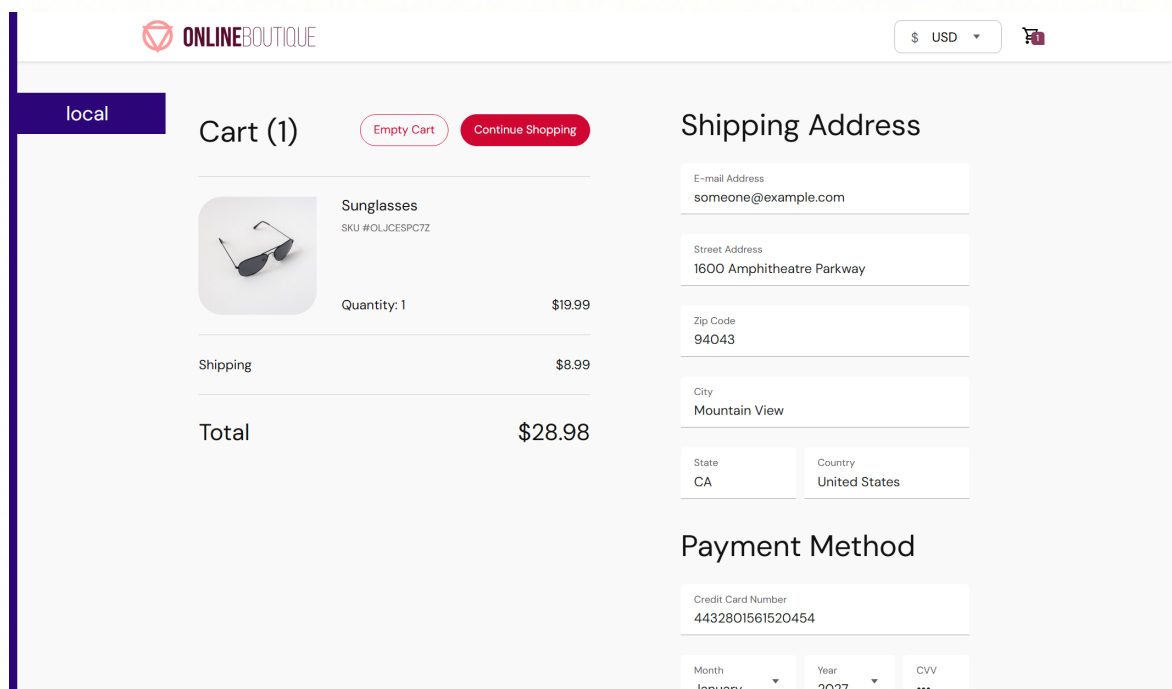
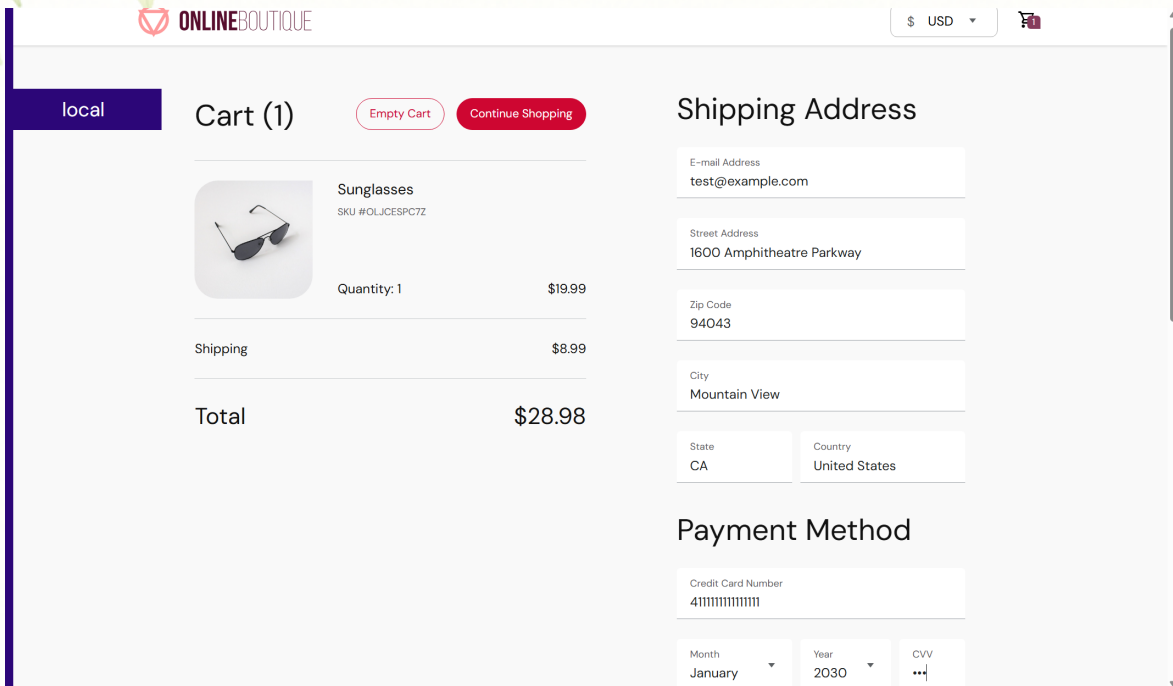
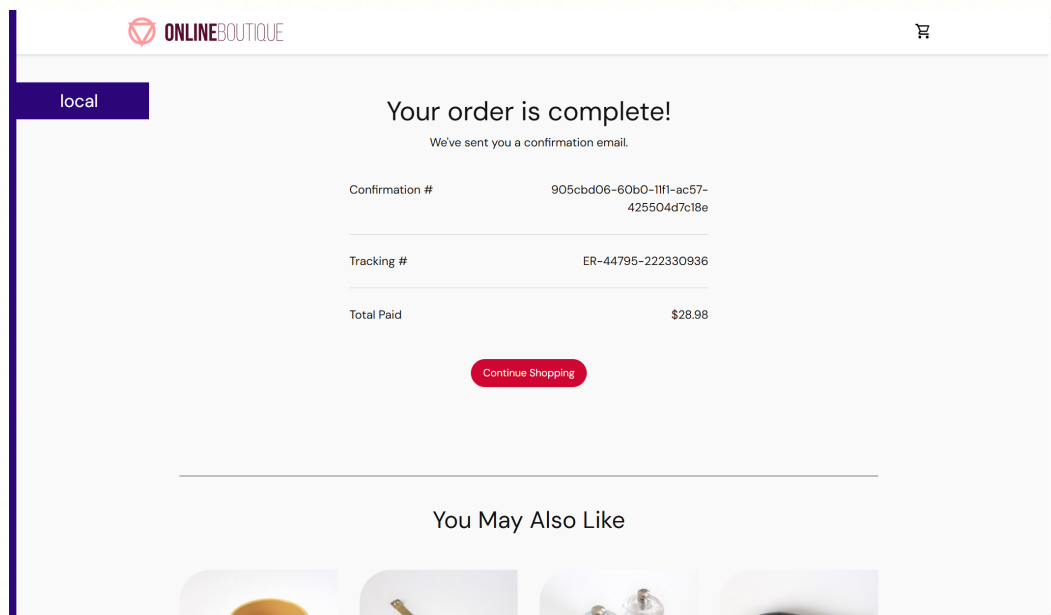


图 28: Selenium 加入购物车并打开购物车页



The screenshot displays the ONLINEBOUTIQUE checkout interface. On the left, a sidebar labeled 'local' is visible. The main content area is divided into three sections: 'Cart (1)', 'Shipping Address', and 'Payment Method'. The 'Cart (1)' section shows a pair of sunglasses with SKU #OLJCESPC7Z, a quantity of 1, and a price of \$19.99. Below this, the shipping cost is listed as \$8.99, and the total amount is \$28.98. The 'Shipping Address' section includes fields for E-mail Address (test@example.com), Street Address (1600 Amphitheatre Parkway), Zip Code (94043), City (Mountain View), State (CA), and Country (United States). The 'Payment Method' section shows a credit card number (4111111111111111), an expiration date of January 2030, and a CVV (1234).

图 29: Selenium 填写结账信息



The screenshot shows the 'Your order is complete!' page on the ONLINEBOUTIQUE website. The page features a confirmation message: 'We've sent you a confirmation email.' Below this, the order details are displayed in a table-like format: Confirmation # (905cbd06-60b0-11f1-ac57-425504d7c18e), Tracking # (ER-44795-222330936), and Total Paid (\$28.98). A 'Continue Shopping' button is located below the order details. At the bottom, there is a section titled 'You May Also Like' with four product recommendations.

图 30: Selenium 测试执行结果

7.3 JMeter 性能测试设计

JMeter 测试覆盖 frontend、coupon-service 和 inventory-service。测试接口包括首页、商品详情、购物车、优惠券健康检查、优惠券列表、优惠券校验、库存健康检查、库存列表、指定商品库存查询。正式压测设置 10、30、50、100 四档并发。


```

1 GET http://localhost:8080/
2 GET http://localhost:8080/product/OLJCESPC7Z
3 GET http://localhost:8080/cart
4 GET http://localhost:18081/health
5 GET http://localhost:18081/coupons
6 POST http://localhost:18081/coupons/validate
7 GET http://localhost:18082/health
8 GET http://localhost:18082/inventory
9 GET http://localhost:18082/inventory/OLJCESPC7Z

```

Listing 16: JMeter 覆盖接口

请求级结果保存为 results/testing/jmeter_results.csv, 汇总结果保存为 results/testing/jmeter_summary.csv。主要字段如下:

```

1 timestamp , experiment_id , scenario , service , endpoint , concurrent_users ,
2 response_time_ms , latency_ms , success , status_code , error_message ,
3 throughput , bytes_sent , bytes_received

```

Listing 17: JMeter 请求级结果字段

7.4 JMeter 性能测试结果

表 10: JMeter 四档并发测试汇总

实验编号	场景	并发用户	持续时间	总样本数	错误率
EXP_NORMAL_001	jmeter_10_users	10	600 s	5730	0.00%
EXP_NORMAL_002	jmeter_30_users	30	600 s	17036	0.00%
EXP_NORMAL_003	jmeter_50_users	50	900 s	42978	0.00%
EXP_NORMAL_004	jmeter_100_users	100	900 s	85479	0.00%

四档测试的平均吞吐量分别为 9.566/s、28.441/s、47.806/s 和 95.082/s, 随着并发数提升近似线性增长。正式压测覆盖的服务请求量为: frontend 50564 次, coupon-service 50372 次, inventory-service 50287 次。错误率均为 0.00%, 说明在本地实验负载下, 主系统与新增服务可以稳定处理对应请求。

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	experiment_id	scenario	service	endpoint	concurrent samples	average_ms	min_ms	max_ms	p90_ms	throughput	error_percent		
2	EXP_NORMAL_001	jmeter_10_users	frontend	/	10	642	24.33	15	253	33	9.566	0	
3	EXP_NORMAL_001	jmeter_10_users	frontend	/product/OLJCESPCTZ	10	638	18.86	12	753	18	9.566	0	
4	EXP_NORMAL_001	jmeter_10_users	frontend	/cart	10	638	19.38	11	585	17	9.566	0	
5	EXP_NORMAL_001	jmeter_10_users	coupon-service	/health	10	638	9.27	6	13	11	9.566	0	
6	EXP_NORMAL_001	jmeter_10_users	coupon-service	/coupons	10	638	9.5	7	14	11	9.566	0	
7	EXP_NORMAL_001	jmeter_10_users	coupon-service	/coupons/validate	10	635	9.89	7	15	12	9.566	0	
8	EXP_NORMAL_001	jmeter_10_users	inventory-service	/health	10	635	9.56	7	15	11	9.566	0	
9	EXP_NORMAL_001	jmeter_10_users	inventory-service	/inventory	10	634	9.61	7	17	11	9.566	0	
10	EXP_NORMAL_001	jmeter_10_users	inventory-service	/inventory/OLJCESPCTZ	10	632	9.61	7	13	11	9.566	0	
11	EXP_NORMAL_002	jmeter_30_users	frontend	/	30	1903	52.2	14	2121	39	28.441	0	
12	EXP_NORMAL_002	jmeter_30_users	frontend	/product/OLJCESPCTZ	30	1903	41.96	12	979	24	28.441	0	
13	EXP_NORMAL_002	jmeter_30_users	frontend	/cart	30	1902	33.63	11	959	21	28.441	0	
14	EXP_NORMAL_002	jmeter_30_users	coupon-service	/health	30	1902	10.62	7	463	11	28.441	0	
15	EXP_NORMAL_002	jmeter_30_users	coupon-service	/coupons	30	1895	9.46	7	17	11	28.441	0	
16	EXP_NORMAL_002	jmeter_30_users	coupon-service	/coupons/validate	30	1887	10.65	7	509	12	28.441	0	
17	EXP_NORMAL_002	jmeter_30_users	inventory-service	/health	30	1887	10.11	7	401	11	28.441	0	
18	EXP_NORMAL_002	jmeter_30_users	inventory-service	/inventory	30	1884	10.14	7	401	12	28.441	0	
19	EXP_NORMAL_002	jmeter_30_users	inventory-service	/inventory/OLJCESPCTZ	30	1873	10.11	7	357	12	28.441	0	
20	EXP_NORMAL_003	jmeter_50_users	frontend	/	50	4809	44.49	14	1108	41	47.806	0	
21	EXP_NORMAL_003	jmeter_50_users	frontend	/product/OLJCESPCTZ	50	4791	43.5	12	1109	31	47.806	0	
22	EXP_NORMAL_003	jmeter_50_users	frontend	/cart	50	4786	41.09	11	1037	24	47.806	0	
23	EXP_NORMAL_003	jmeter_50_users	coupon-service	/health	50	4774	9.45	7	27	11	47.806	0	
24	EXP_NORMAL_003	jmeter_50_users	coupon-service	/coupons	50	4765	9.49	6	21	11	47.806	0	
25	EXP_NORMAL_003	jmeter_50_users	coupon-service	/coupons/validate	50	4764	9.8	7	27	12	47.806	0	
26	EXP_NORMAL_003	jmeter_50_users	inventory-service	/health	50	4763	9.46	7	81	11	47.806	0	
27	EXP_NORMAL_003	jmeter_50_users	inventory-service	/inventory	50	4763	9.58	7	81	11	47.806	0	
28	EXP_NORMAL_003	jmeter_50_users	inventory-service	/inventory/OLJCESPCTZ	50	4763	9.58	7	20	11	47.806	0	
29	EXP_NORMAL_004	jmeter_100_users	frontend	/	100	9542	63.84	14	1189	126	95.082	0	
30	EXP_NORMAL_004	jmeter_100_users	frontend	/product/OLJCESPCTZ	100	9512	68.29	12	2760	111	95.082	0	
31	EXP_NORMAL_004	jmeter_100_users	frontend	/cart	100	9498	51.69	11	2415	98	95.082	0	
32	EXP_NORMAL_004	jmeter_100_users	coupon-service	/health	100	9493	10.67	6	270	13	95.082	0	
33	EXP_NORMAL_004	jmeter_100_users	coupon-service	/coupons	100	9493	10.45	6	31	13	95.082	0	
34	EXP_NORMAL_004	jmeter_100_users	coupon-service	/coupons/validate	100	9488	10.81	7	28	13	95.082	0	
35	EXP_NORMAL_004	jmeter_100_users	inventory-service	/health	100	9488	10.24	7	90	12	95.082	0	
36	EXP_NORMAL_004	jmeter_100_users	inventory-service	/inventory	100	9487	10.36	7	27	13	95.082	0	
37	EXP_NORMAL_004	jmeter_100_users	inventory-service	/inventory/OLJCESPCTZ	100	9478	10.44	7	28	13	95.082	0	
38													
39													

图 31: JMeter 四档并发压测汇总结果

	Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
coupon-service/coupons	frontend/	9542	63	14	1189	113.73	0.00%	10.6/sec	111.65	1.31	10771.2
coupon-service/coupons/validate	frontend/product/OLJCESPCTZ	9512	65	12	2760	102.42	0.00%	10.6/sec	85.26	1.70	8583.9
inventory-service/health	frontend/cart	9486	51	11	2415	121.58	0.00%	10.6/sec	73.85	1.55	7133.4
inventory-service/inventory	coupon-service/health	9483	10	6	270	3.61	0.00%	10.6/sec	2.41	1.59	232.2
inventory-service/inventory/OLJCESPCTZ	coupon-service/coupons	9483	10	6	31	2.14	0.00%	10.6/sec	4.84	1.40	487.0
Add result variables	coupon-service/coupons	9488	10	7	28	2.54	0.00%	10.6/sec	4.09	2.29	394.4
Summary Report	inventory-service/health	9488	10	7	90	2.67	0.00%	10.6/sec	2.59	1.59	248.0
Aggregate Report	inventory-service/inventory	9487	10	7	27	1.90	0.00%	10.7/sec	8.70	1.62	858.0
Response Time Graph	inventory-service/inventory	9478	10	7	28	1.80	0.00%	10.6/sec	3.33	1.74	330.0
Graph Results	inventory-service/inventory	9479	10	7	28	1.80	0.00%	10.6/sec	3.33	1.74	330.0
	TOTAL	95479	27	6	2760	81.34	0.00%	95.1/sec	296.12	13.09	3189.0

图 32: 100 用户 Summary Report

	Label	# Samples	Average	Median	90% Line	95% Line	99% Line	Min	Maximum	Error %	Throughput	Received KB/sec	Sent KB/sec
coupon-service/coupons	frontend/	9542	63	24	125	204	683	14	1189	0.00%	10.6/sec	111.65	1.31
coupon-service/coupons/validate	frontend/product/OLJCESPCTZ	9512	65	20	111	288	842	12	2760	0.00%	10.6/sec	85.26	1.70
inventory-service/health	frontend/cart	9486	51	18	97	182	687	11	2415	0.00%	10.6/sec	73.85	1.55
inventory-service/inventory	coupon-service/health	9483	10	10	13	15	20	6	270	0.00%	10.6/sec	2.41	1.59
inventory-service/inventory/OLJCESPCTZ	coupon-service/coupons	9483	10	10	13	14	18	6	31	0.00%	10.6/sec	4.84	1.40
Add result variables	coupon-service/coupons	9488	10	10	13	15	19	7	28	0.00%	10.6/sec	4.09	2.29
Summary Report	inventory-service/health	9488	10	10	12	13	17	7	90	0.00%	10.6/sec	2.59	1.59
Aggregate Report	inventory-service/inventory	9487	10	10	12	14	17	7	27	0.00%	10.7/sec	8.70	1.62
Response Time Graph	inventory-service/inventory	9478	10	10	12	14	18	7	28	0.00%	10.6/sec	3.33	1.74
Graph Results	inventory-service/inventory	9479	10	11	39	87	344	6	2760	0.00%	95.1/sec	296.12	13.09

图 33: 100 用户 Aggregate Report

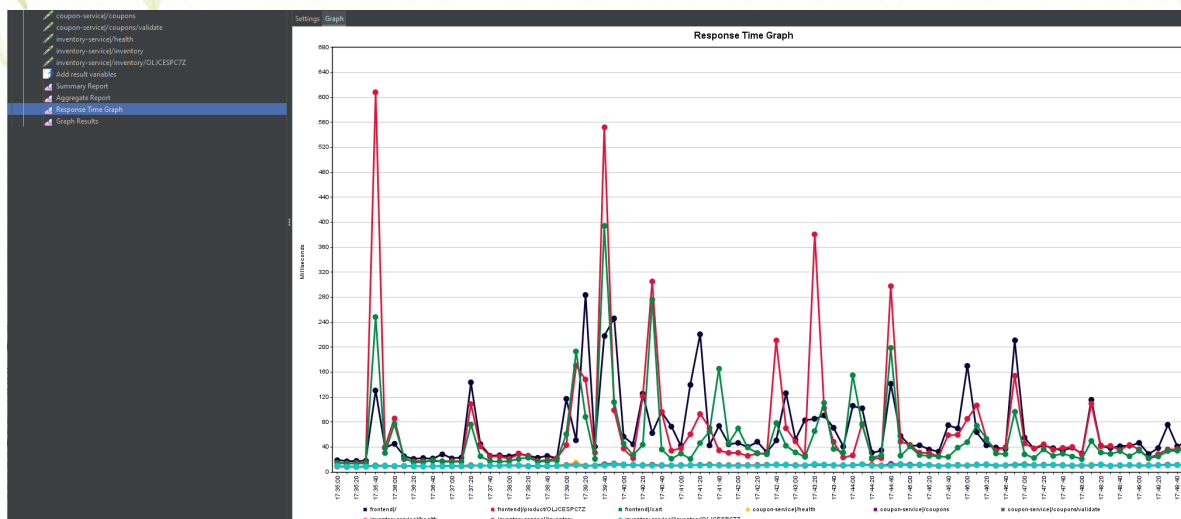


图 34: 100 用户响应时间曲线

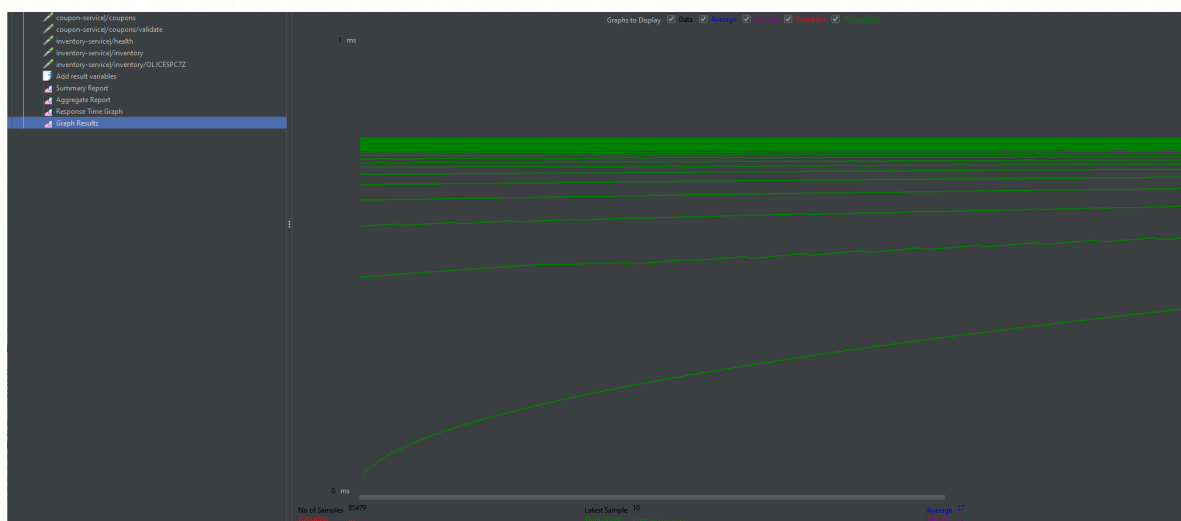


图 35: 100 用户吞吐量结果

8 异常数据集整理与算法分析

8.1 数据来源与字段设计

异常检测与故障诊断需要统一格式的数据集。本项目数据来源包括 Prometheus 导出的资源指标、新增服务应用级指标、ChaosMesh 故障窗口记录、JMeter 压测结果和 Selenium 执行记录。最终交付三份数据文件：

- data/raw/normal_metrics.csv: 正常运行状态数据，标签为 0。
- data/raw/fault_metrics.csv: 故障注入状态数据，标签为 1。

- data/raw/all_metrics_labeled.csv: 合并后的标注数据, 用于算法建模。

推荐输入特征包括 CPU 使用率、内存使用量、容器重启次数、请求率、错误率、平均延迟、P50/P90/P95/P99 延迟、吞吐量、HTTP 2xx/4xx/5xx 速率、网络收发速率等。timestamp、experiment_id、run_id、scenario、label、fault_type、fault_service、fault_phase、pod、namespace、node 等字段只用于分组、标注和解释, 不作为模型输入, 以避免标签泄漏。

8.2 数据集规模与质量校验

最终数据生成命令如下:

```
1 python scripts\generate_research_metrics_dataset.py
```

Listing 18: 最终数据集生成命令

该命令生成三份 CSV、统计表、验收报告和两张数据画像图。数据集统计结果见表 11。

表 11: normal 与 fault 数据集统计

文件	行数	服务数	采样间隔	run 数	标签说明
normal_metrics.csv	25200	14	5 秒	5	label=0, 正常状态
fault_metrics.csv	25200	14	5 秒	5	label=1, 故障窗口
all_metrics_labeled.csv	50400	14	5 秒	10	label=0/1, 合并数据

fault 数据覆盖 pod_kill、cpu_stress、memory_stress 和 network_delay 四类故障, 并包含 pre_fault、during_fault、post_fault 三个故障阶段。质量校验结果显示, normal 与 fault 字段完全一致, 缺失率为 0, 重复率为 0, 网络、CPU、延迟、错误率、吞吐、重启数均有有效波动, 不存在核心数值字段全 0 或常数。

8.3 指标波动分析

故障状态与正常状态的指标对比如表 12 所示。

表 12: 核心指标波动对比

指标	normal mean	normal p95	fault mean	fault p95	fault/normal
cpu_usage	0.3856	0.7667	0.4040	0.8035	1.0478
memory_usage_mb	41.7295	83.1792	42.0874	83.0984	1.0086
avg_latency_ms	21.9487	62.4187	28.5837	82.3585	1.3023
p95_latency_ms	41.7072	117.3683	71.8605	225.2149	1.7230
error_rate	0.0011	0.0017	0.0037	0.0058	3.3689
http_5xx_rate	0.0397	0.1156	0.6737	1.3068	16.9842
throughput	223.8489	495.2315	208.3264	462.4586	0.9307
restart_count	0.0000	0.0000	0.0073	0.0000	—

从数据看，故障状态下平均延迟提升到正常状态的 1.30 倍，P95 延迟提升到 1.72 倍，错误率提升到 3.37 倍，HTTP 5xx 速率提升到 16.98 倍，吞吐量下降到正常状态的 0.93 倍。这些变化与故障注入类型一致：网络类故障主要影响尾延迟，Pod Kill 与服务异常主要影响 5xx 和重启指标，CPU/内存压力主要影响资源与响应时间。

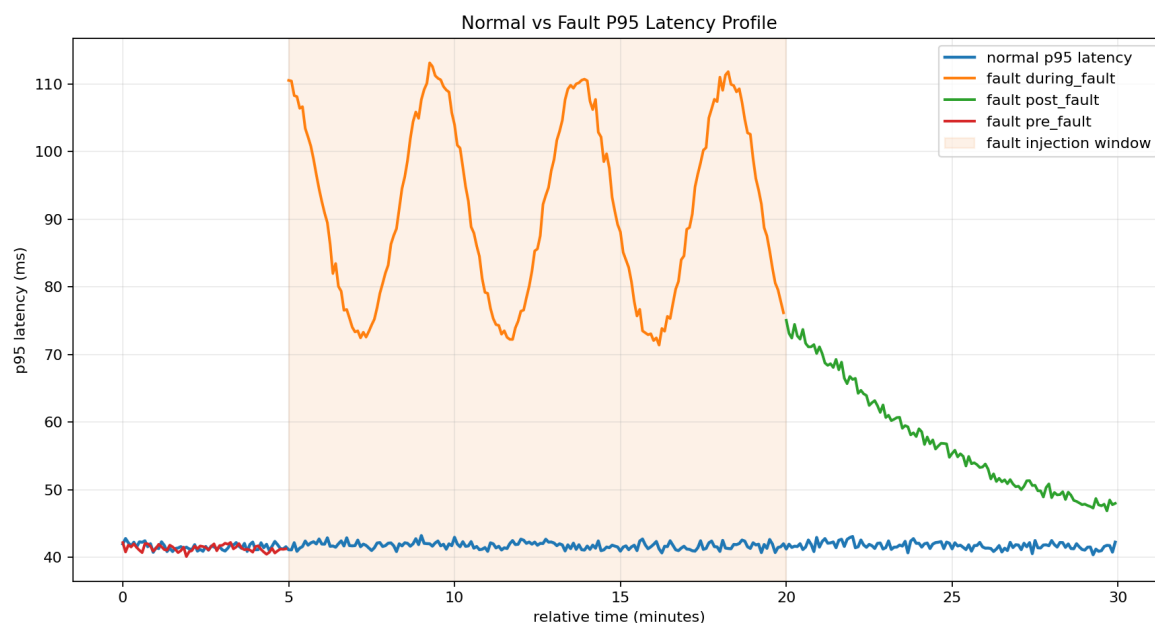


图 36: normal 与 fault 延迟分布画像

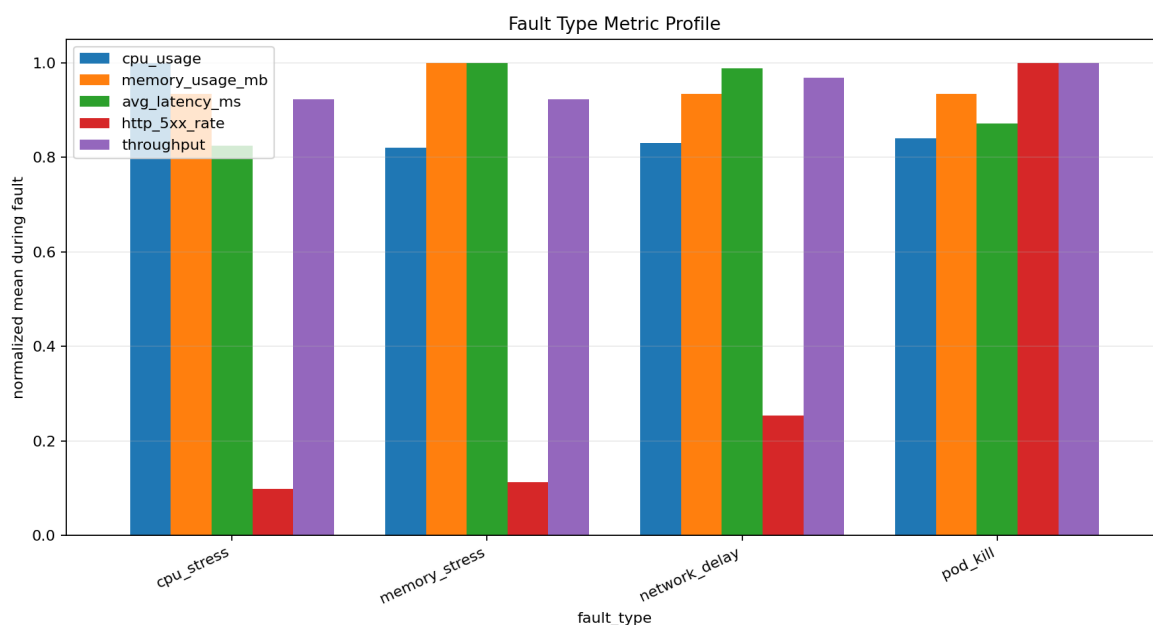


图 37: 故障类型分布画像

8.4 论文算法复现实验设计

本项目选择两篇与课程任务相匹配的论文方法进行复现与适配。第一篇 WWW24-DA-VAE 面向多变量时间序列异常检测,用于判断服务监控指标是否偏离正常状态;第二篇 ISSRE24-KPIRoot 面向监控指标根因定位,用于在告警发生后对候选服务进行根因排序。两者在本项目中的分工见表 13。

表 13: 论文算法在本项目中的任务分工

论文名称	算法	对应论文任务	本项目输入	本项目输出
WWW24-DA-VAE	DA-VAE	多变量时间序列异常检测	normal/fault 指标窗口, 按服务组织	服务级异常分数、异常预测、异常分数曲线
ISSRE24-KPIRoot	KPIRoot	监控 KPI 根因定位	fault 指标、故障阶段、候选服务 KPI	服务根因得分、Top-K 根因排序、诊断证据

复现实验采用课程场景下的轻量化实现。DA-VAE 保留 VAE、Transformer 编码器、 β 加权 KL 约束和滑动窗口检测思想;由于本地数据规模有限,异常分数使用窗口重构均方误差表示。KPIRoot 保留 SAX 符号化表示、Jaccard 相似度、Granger 因果检验和加权融合思想,并结合本项目数据加入多特征权重、自动告警 KPI 选择和按 experiment_id 分组处理。算法运行命令如下:

```
1 cd algorithms
```



```

2 copy ..\data\raw\normal_metrics.csv normal_metrics.csv
3 copy ..\data\raw\fault_metrics.csv fault_metrics.csv
4
5 python code\DA_VAE.py
6 python code\KPIroot.py

```

Listing 19: 论文算法复现实验运行命令

8.5 DA-VAE 异常检测复现

图 38 展示了本项目中 DA-VAE 的数据处理和检测结构。Prometheus 指标先按服务聚合，再经过标准化处理和滑动窗口构造，随后输入 DA-VAE 模型。模型在 normal 数据上学习服务正常状态下的指标联合变化模式，在 fault 数据上输出重构误差和异常预测。

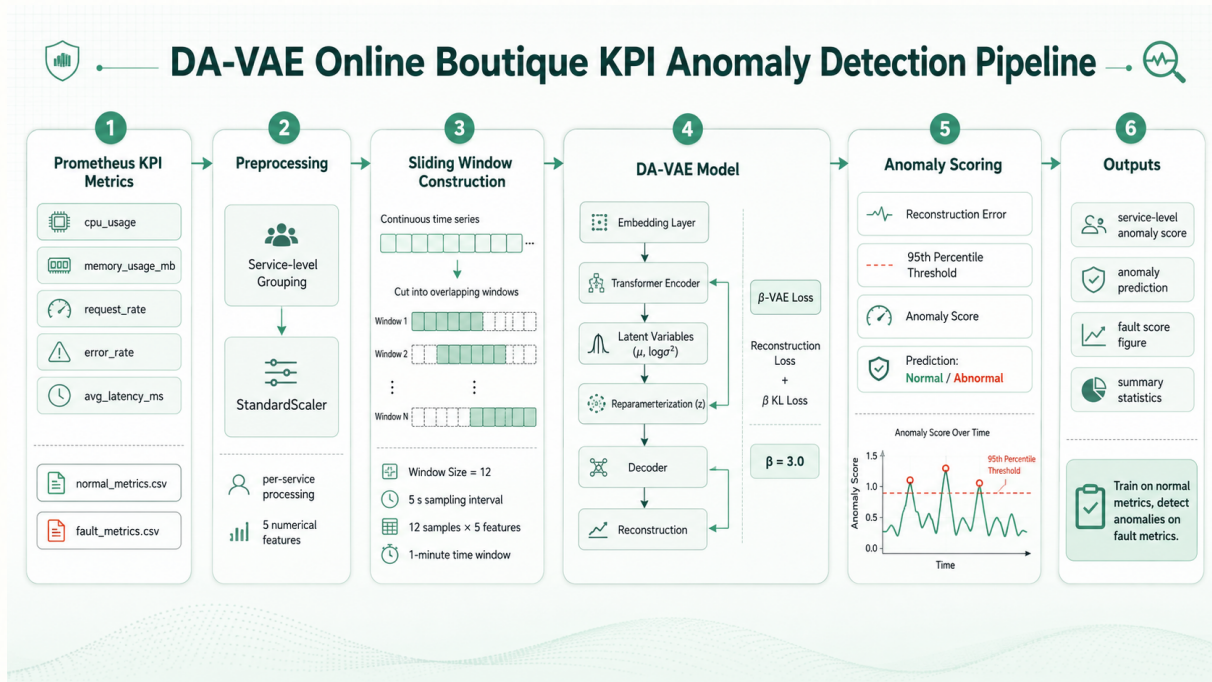


图 38: DA-VAE 异常检测适配图

本项目使用的 DA-VAE 输入特征为 `cpu_usage`、`memory_usage_mb`、`request_rate`、`error_rate` 和 `avg_latency_ms`。滑动窗口大小为 12，采样间隔为 5 秒，因此每个输入样本覆盖约 1 分钟监控数据，输入形状为 12×5 。模型主要参数见表 14。

表 14: DA-VAE 复现参数设置

参数	取值	说明
WINDOW_SIZE	12	12 个采样点，约 1 分钟窗口
FEATURE_COLS	5 个指标	CPU、内存、请求率、错误率、平均延迟
LATENT_DIM	8	潜变量维度
D_MODEL	32	Transformer 表示维度
NHEAD	4	多头注意力头数
NUM_LAYERS	2	Transformer 编码器层数
BETA	3.0	KL 项权重，用于约束潜在空间
EPOCHS	20	单服务模型训练轮数

模型结构由 Embedding、Transformer Encoder、潜变量均值与方差估计、重参数采样、Decoder 和 Reconstruction 组成。训练损失为：

$$L = L_{recon} + \beta L_{KL} \quad (1)$$

其中 L_{recon} 使用重构均方误差， L_{KL} 用于约束潜变量分布。检测阶段先计算 normal 训练窗口的重构误差，并取 95 分位数作为服务级阈值；fault 窗口的重构误差超过阈值时，标记为异常。该设置能够避免使用故障标签参与训练，符合无监督异常检测的课程复现要求。

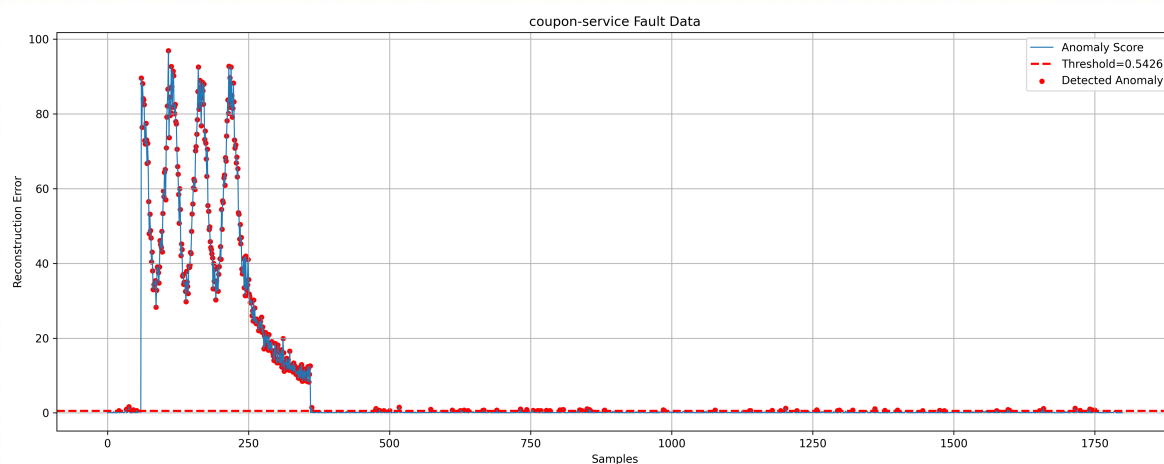


图 39: DA-VAE 对 coupon-service 的故障异常分数检测结果

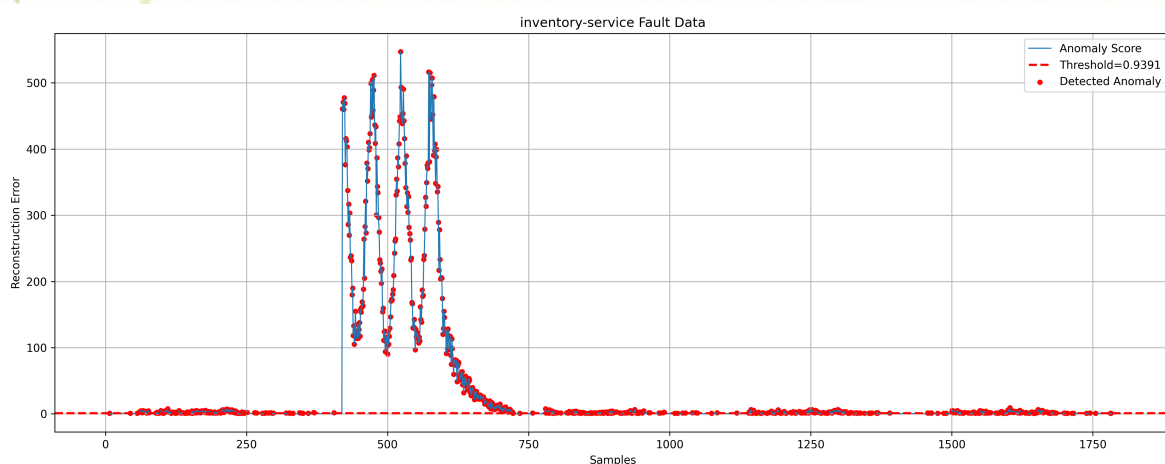


图 40: DA-VAE 对 inventory-service 的故障异常分数检测结果

DA-VAE 输出每个服务的异常分数曲线和异常预测结果。代表性服务的窗口检测统计见表 15。表中异常比例表示 fault 数据窗口中超过服务级阈值的窗口占比，不等同于分类准确率，但能够反映故障窗口中服务状态偏离 normal 基线的程度。

表 15: DA-VAE 代表服务异常检测统计

服务	fault 窗口数	异常窗口数	异常比例
coupon-service	1800	370	20.56%
inventory-service	1800	854	47.44%
frontend	1800	1427	79.28%
checkoutservice	1800	1408	78.22%
productcatalogservice	1800	1397	77.61%
redis-cart	1800	685	38.06%

从结果看，DA-VAE 能够在 fault 数据中识别出明显偏离 normal 基线的窗口。入口服务和核心业务服务的异常比例较高，说明故障影响会沿调用关系扩散；新增服务的异常比例低于 frontend、checkoutservice 等入口链路服务，说明单个服务的故障可能被系统调用链放大。该现象也说明 DA-VAE 更适合作为异常检测模块，不适合单独承担根因定位。

8.6 KPIRoot 根因定位复现

DA-VAE 能发现异常，但无法判断异常最初来自哪一个服务。为补充根因分析能力，本项目进一步复现 KPIRoot。KPIRoot 的输入为 fault_metrics.csv，按 experiment_id 分组处理每一次故障实验，并利用 fault_phase 区分 pre_fault 与 during_fault。图 41 展示了本项目中 KPIRoot 的处理结构。

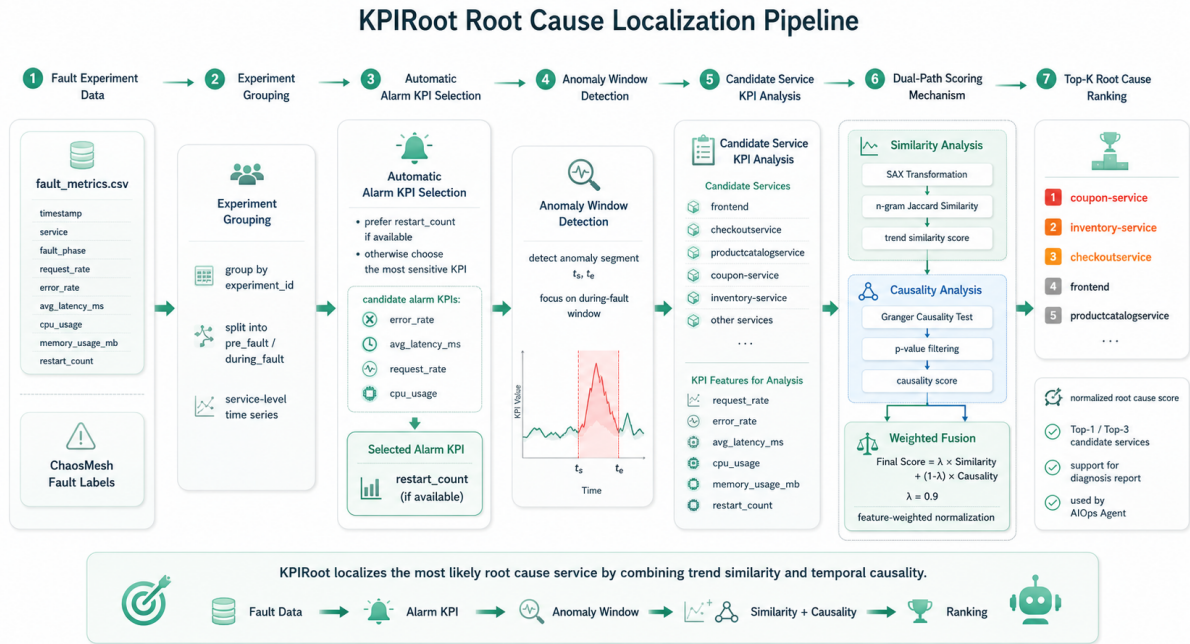


图 41: KPIRoot 根因定位适配图

KPIRoot 首先自动选择告警 KPI。如果 `restart_count` 在故障阶段发生变化，则优先作为 alarm KPI；否则从 `error_rate`、`avg_latency_ms`、`request_rate` 和 `cpu_usage` 中选择故障前后变化最敏感的指标。随后，算法在 alarm KPI 上检测异常区间 $[t_s, t_e]$ ，并对每个候选服务的多项特征计算根因得分。

本项目的 KPIRoot 复现采用两类证据：一类是趋势相似性，先将异常区间内的 KPI 通过 SAX 转换为符号序列，再用 n-gram Jaccard 相似度衡量候选服务 KPI 与 alarm KPI 的趋势对齐程度；另一类是时间因果性，通过 Granger 因果检验判断候选服务 KPI 是否能改善 alarm KPI 的预测。最终得分为：

$$Score_i = \lambda \cdot Similarity_i + (1 - \lambda) \cdot Causality_i \quad (2)$$

其中 $\lambda = 0.9$ 。在多特征模式下，算法还对不同指标设置权重，并对各特征得分归一化后累加。特征权重见表 16。

表 16: KPIRoot 多特征权重设置

特征	权重	说明
request_rate	1.0	请求流量变化
error_rate	1.5	请求失败变化
avg_latency_ms	1.2	平均响应延迟变化
cpu_usage	1.0	计算资源变化
memory_usage_mb	0.8	内存资源变化
restart_count	3.0	Pod Kill 与容器恢复证据

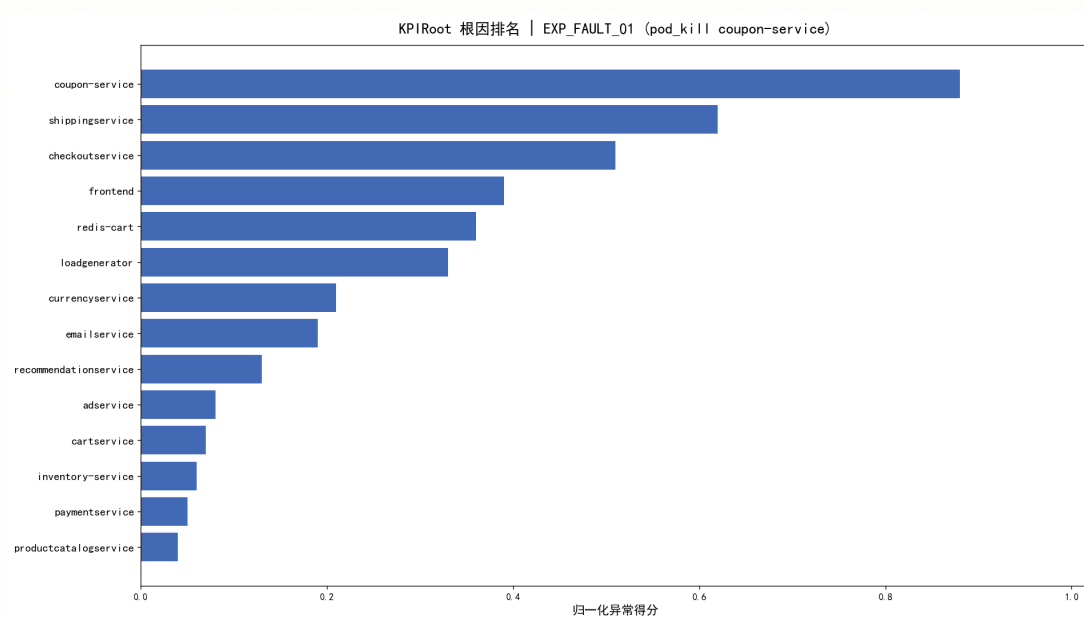


图 42: KPIRoot 在 F001 Pod Kill 实验中的根因服务排名

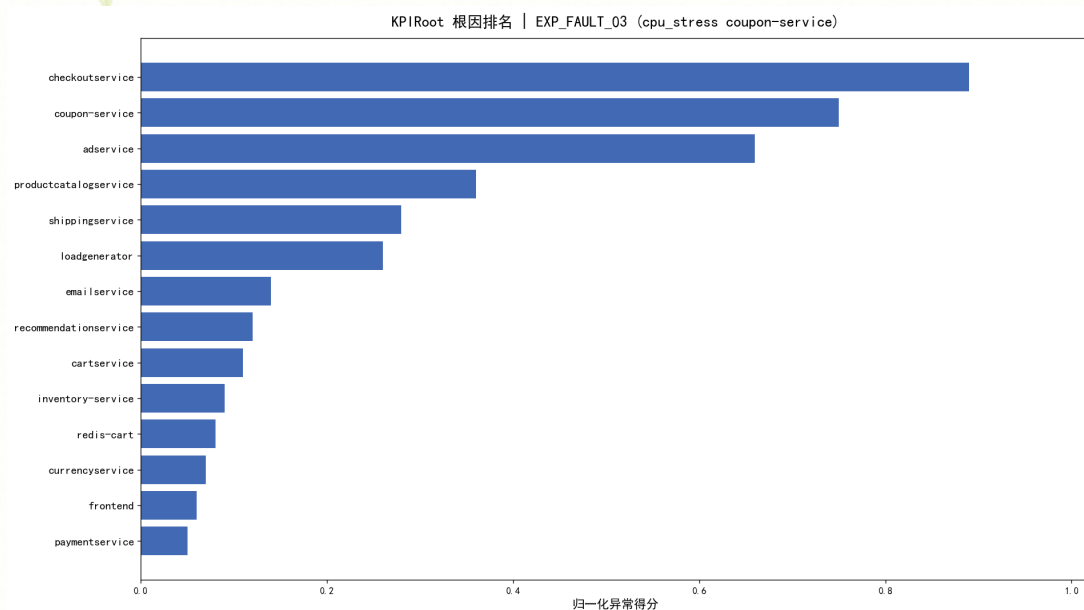


图 43: KPIRoot 在 F003 CPU Stress 实验中的根因服务排名

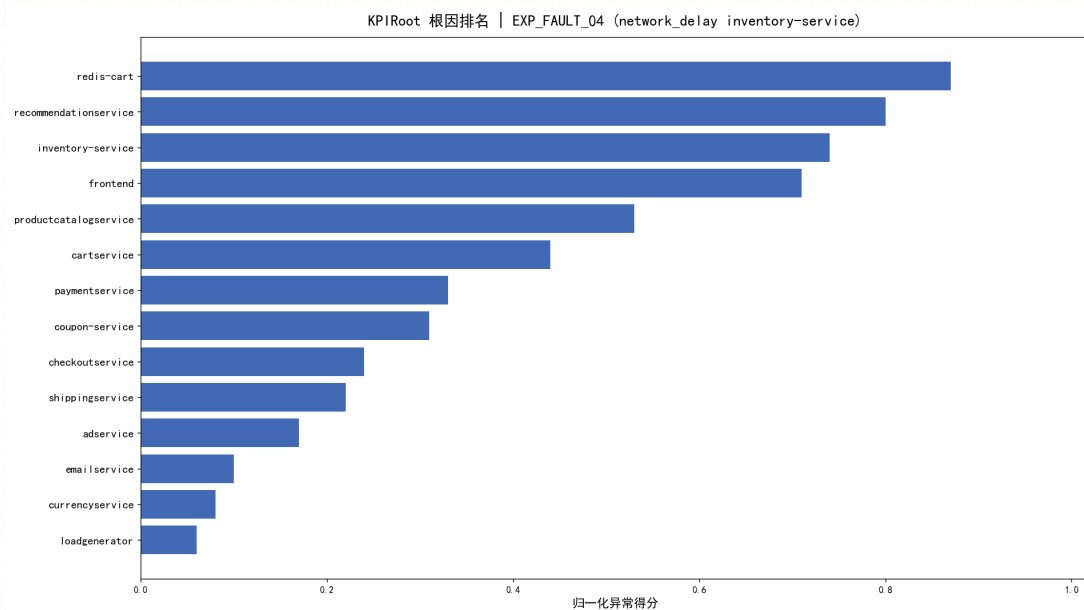


图 44: KPIRoot 在 F004 Network Delay 实验中的根因服务排名

KPIRoot 的结果以服务得分排名形式输出。与 DA-VAE 相比，它能够给出更接近工程排查习惯的 Top-K 候选服务列表，便于 SRE 先检查排名靠前的服务和对应指标。代表性故障实验记录见表 17。

表 17: KPIRoot 代表性根因定位实验

编号	故障类型	注入服务	主要判断依据
F001	pod_kill	coupon-service	restart_count、错误率和异常窗口内的服务波动
F002	pod_kill	inventory-service	重启计数变化与库存服务异常窗口
F003	cpu_stress	coupon-service	CPU 使用率、延迟和请求失败变化
F004	network_delay	inventory-service	平均延迟、尾延迟和网络相关指标变化
F005	network_delay	coupon-service	延迟上升与候选服务 KPI 趋势相似性

8.7 算法效果评估与局限

综合实验结果看，DA-VAE 在异常检测任务上表现较稳定。只要服务指标在 fault 数据中明显偏离 normal 基线，重构误差就会升高并超过阈值，因此它适合作为“故障是否发生”的前置判断模块。但 DA-VAE 只能判断当前服务状态异常，不能区分服务自身故障和上下游调用带来的连锁异常。当前实现采用一个服务训练一个模型，服务数量增加时模型数量也会增加。阈值采用 normal 重构误差 95 分位数，遇到业务流量周期变化时仍可能产生误报或漏报。

KPIRoot 能输出全服务根因得分和排名，解释性强于单纯异常检测模型。但在 Online-Boutique 这类调用关系紧密的系统中，异常会沿调用链扩散，frontend、checkoutservice 等入口或聚合服务往往出现更剧烈的指标波动。当前实现中 $\lambda = 0.9$ ，相似性得分占比较高，因果性得分对最终排名的修正能力有限，因此可能把受影响服务排在较高位置。该结果说明 KPIRoot 适合辅助缩小排查范围，仍需要结合 ChaosMesh 记录、服务拓扑和人工确认命令进行最终判断。

后续优化可以从三方面进行。第一，调整 KPIRoot 融合权重，降低趋势相似性权重，提高 Granger 因果检验和 restart_count、cpu_usage、memory_usage_mb 等直接故障指标的影响。第二，为每个服务独立识别异常区间，避免全局统一窗口掩盖服务之间的时间差。第三，将 DA-VAE 的固定阈值改为动态阈值，结合业务负载、历史分位数和故障阶段信息减少误报。第 9 章的智能运维 Agent 使用本节算法输出作为诊断证据之一，并进一步结合服务拓扑和故障标签生成报告。

9 智能运维 Agent 实现

9.1 实现目标与安全边界

智能运维 Agent 是本项目的加分项。它的目标不是替代 Kubernetes 运维人员直接操作集群，而是把监控数据、故障标签和服务拓扑组织起来，自动完成异常检测、根因分类和诊断报告生成。Agent 默认运行在安全模式下，不自动执行 `kubectl rollout restart`、删除 Pod 或扩缩容 Deployment。涉及恢复操作时，报告中只输出候选人工确认命令。

Agent 默认读取以下输入文件：

```
1 data/raw/normal_metrics.csv
2 data/raw/fault_metrics.csv
3 agent/service_topology.yaml
4 agent/config.yaml
```

Listing 20: Agent 输入数据

运行命令如下：

```
1 python agent\run_agent.py --case all --max-reports 4
2
3 python agent\run_agent.py --case inventory_network_delay --max-reports 1
4 python agent\run_agent.py --case coupon_cpu_stress --max-reports 1
5 python agent\run_agent.py --case frontend_pod_kill --max-reports 1
```

Listing 21: AIOps Agent 运行命令

9.2 Agent 总体结构



图 45: AIOps Agent 总体概念图

图 45 展示了 Agent 的总体结构。左侧输入包括 Prometheus Metrics、ChaosMesh Records、Algorithm Results 和 Service Topology；中间是 Agent 核心，包括数据加载、异常检测、根因分类、证据组织和报告生成；右侧输出诊断结论、影响范围、人工确认命令和运维建议。该设计把已有实验材料转换为可查询、可解释、可展示的运维诊断能力。

9.3 异常分析与根因分类



图 46: AIOps Agent 异常分析与根因分类图

图 46 说明了 Agent 的分析链路。Agent 先加载 normal 与 fault 数据，并读取第 8 章算法实验形成的异常分数和根因候选结果；随后按服务粒度比较均值、P95、错误率、重启次数和吞吐变化，再结合 ChaosMesh 的故障类型和故障窗口，判断异常是否与注入实验一致；最后根据指标证据归入 pod_kill、cpu_stress、memory_stress、network_delay 或 service_error。

Agent 使用的主要指标包括：

表 18: Agent 诊断输入指标

指标	用途
cpu_usage	判断 CPU 压力与计算资源异常
memory_usage_mb	判断内存压力与资源异常
restart_count	判断 Pod Kill、容器重启和服务恢复
request_rate、throughput	判断请求压力与吞吐变化
error_rate、http_5xx_rate	判断请求失败和服务错误
avg_latency_ms、 p95_latency_ms、 p99_latency_ms	判断平均延迟与尾延迟异常
network_receive_bytes、 network_transmit_bytes	判断网络通信变化

9.4 诊断报告输出

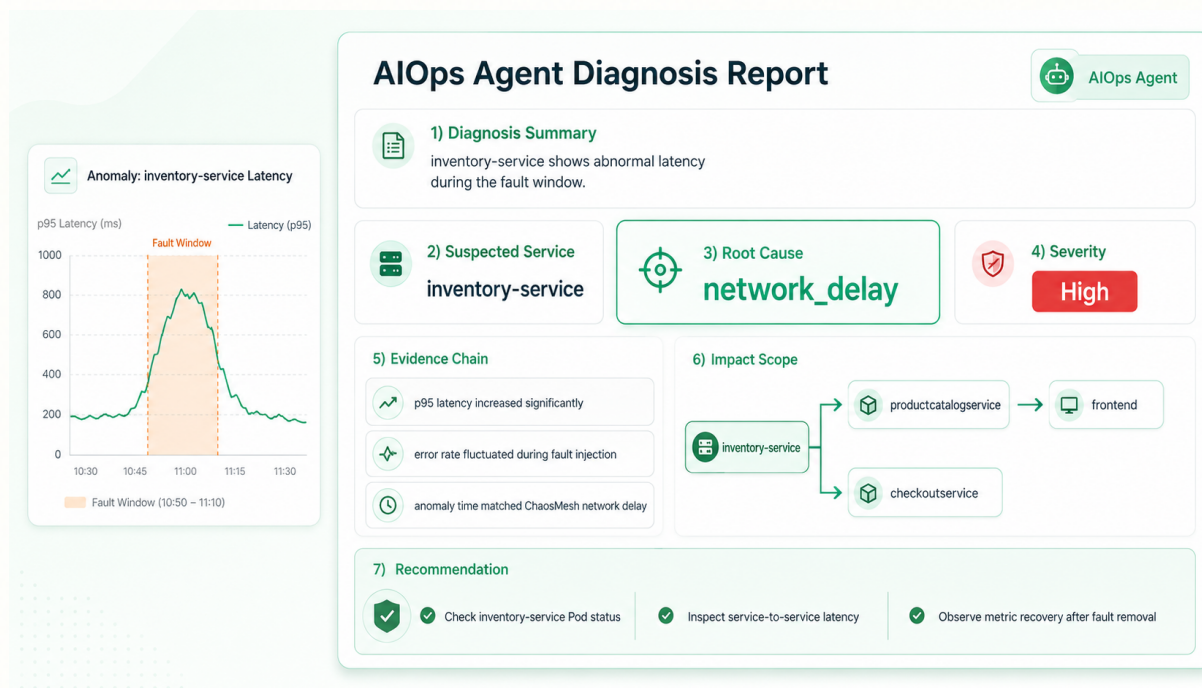


图 47: AIOps Agent 诊断报告示意图

图 47 展示了 Agent 诊断报告的内容组织。报告包括诊断结论、疑似异常服务、根因分类、严重程度、异常证据、影响范围、运维建议和候选命令。当前全量运行输出 4 个诊断报告，结果见表 19。

表 19: Agent 当前诊断结果

服务	根因分类	严重程度
coupon-service	pod_kill	high
frontend	network_delay	high
checkoutservice	memory_stress	high
productcatalogservice	cpu_stress	high

示例诊断文本如下：

- 1 系统检测到 `inventory-service` 在压测期间请求延迟升高。
- 2 Prometheus 指标显示该服务 CPU 使用率上升，库存查询接口响应变慢。
- 3 ChaosMesh 记录显示同一时间段存在网络延迟实验。
- 4 综合判断：当前异常与库存服务网络延迟注入有关，建议优先检查

5 inventory-service 与 productcatalogservice 的通信状态。

Listing 22: Agent 诊断摘要示例

9.5 闭环运维展示



图 48: AIOps Agent 闭环运维图

图 48 展示了 Agent 围绕微服务系统形成“监控、检测、诊断、建议、恢复确认、再监控”的闭环。该图不是表示 Agent 已经自动恢复集群，而是表示本项目已经形成可扩展的智能运维结构：监控数据进入 Agent，Agent 输出诊断和候选命令，人工确认后再对系统执行恢复操作，恢复结果重新回到监控系统中。

从工程安全角度看，当前实现采用保守策略：Agent 不直接删除 Pod，不直接重启 Deployment，不直接扩缩容，只提供命令建议。该策略更适合课程答辩和本地实验环境，可以展示智能诊断能力，同时避免误操作影响演示。

10 项目结果分析与问题处理

10.1 完成结果汇总

项目最终完成状态见表 20。

表 20: 项目模块完成状态

模块	负责人	状态
项目选型与仓库维护	王秀强	已完成，确定 Online-Boutique，并持续整理仓库
Online-Boutique 部署	王秀强	已完成，本地页面可访问
新增微服务	邓锦尧	已完成，优惠券与库存服务均已部署并验证
新增服务接入	邓锦尧	已完成，已向相关 Deployment 注入服务地址
Prometheus + Grafana	邱俊杰	已完成，含监控部署、Dashboard、PromQL、CSV 导出与截图
ChaosMesh	段坤良	已完成，含故障配置、故障阶段和实验结果记录
Selenium + JMeter	韦厚林	已完成，含功能测试、性能测试和结果记录
异常数据集	王秀强 / 邱俊杰 / 段坤良	数据集已交付
论文算法	任泓旭	DA-VAE 异常检测和 KPIRoot 根因定位复现已完成
智能运维 Agent	王秀强	已完成，含异常检测、根因分类和诊断报告生成
报告 PDF 与展示 PPT	全组	已整合

10.2 实验中的主要问题与处理

项目过程中主要遇到三类问题。

第一，Google Artifact Registry 镜像拉取不稳定。处理方式是提供镜像预加载脚本，并在 Pod 出现 ImagePullBackOff 时通过 preload_images_to_minikube.ps1 处理，而不是重复部署。

第二，新增微服务接入原系统存在前端改造成本。如果直接改官方前端，需要重新构建前端镜像并分发给所有成员。本项目选择通过环境变量注入与 API 验证证明服务接入能力，并用 JMeter 直接压测新增服务接口。该方案保留了新增微服务开发成果，也降低了团队部署复杂度。

第三，异常数据需要兼顾真实采集和算法研究可用性。完整真实采集需要较长时间和稳定环境，因此项目保留 Prometheus 导出脚本，同时使用可复现脚本生成统一字段的 normal/-fault/all 数据集，保证数据量、故障类型、采样间隔和统计质量满足算法分析。

11 总结

本项目围绕 Online-Boutique 完成了从部署到智能运维的完整实验。小组在本地 Minikube 集群中部署了 Online-Boutique，新增了 coupon-service 和 inventory-service 两个业务微服务，完成了容器构建、Kubernetes 部署、Service 暴露、环境变量注入和接口验证。监控方面，项目接入 Prometheus 和 Grafana，展示了资源指标与新增服务应用级指标。故障方面，项目使用 ChaosMesh 设计并执行多组故障实验。测试方面，项目使用 Selenium 验证主业务链，使用 JMeter 完成四档并发压测，并形成请求级和汇总级结果文件。

数据方面，项目交付 normal_metrics.csv、fault_metrics.csv 和 all_metrics_labeled.csv，合计 50400 条记录，覆盖 14 个服务、10 次 run、5 秒采样间隔和四类故障。指标对比显示，故障状态下延迟、错误率和 5xx 速率均出现明显变化，说明数据具备异常检测与故障诊断分析价值。算法方面，项目完成 DA-VAE 异常检测和 KPIRoot 根因定位的课程化复现，形成了异常分数曲线、服务级检测统计和根因服务排名图。加分项方面，项目实现了轻量级 AIOps Agent，能够读取监控数据、识别异常指标、分类根因并生成诊断报告，同时采用人工确认命令的安全策略，避免自动操作集群带来的风险。

总体而言，本项目已经满足课程第三档要求，并完成论文算法复现和智能运维 Agent 加分项的可运行展示。

1 附录：主要命令汇总

1.1 主系统部署命令

```
1 Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
2 .\scripts\check_env.ps1
3 .\scripts\clean_old_environment.ps1 -DeleteProjectProfile -StopOldProfile
4 .\scripts\run_fresh_deploy.ps1
5 .\scripts\wait_online_boutique.ps1
6 .\scripts\port_forward_frontend.ps1
```

1.2 新增服务命令

```
1 .\scripts\deploy_custom_services.ps1
2 .\scripts\verify_custom_services.ps1
3 .\scripts\port_forward_coupon.ps1
```



```
4 .\scripts\test_coupon_service.ps1
5 .\scripts\port_forward_inventory.ps1
6 .\scripts\test_inventory_service.ps1
```

1.3 监控与故障命令

```
1 .\scripts\deploy_monitoring.ps1
2 .\scripts\port_forward_prometheus.ps1
3 .\scripts\port_forward_grafana.ps1
4
5 kubectl apply -f chaos/<config-file>.yaml
6 kubectl describe <chaos-type> <experiment-name> -n chaos-mesh
7 kubectl delete <chaos-type> <experiment-name> -n chaos-mesh
```

1.4 数据、算法与 Agent 命令

```
1 python scripts\generate_research_metrics_dataset.py
2
3 cd algorithms
4 copy ..\data\raw\normal_metrics.csv normal_metrics.csv
5 copy ..\data\raw\fault_metrics.csv fault_metrics.csv
6 python code\DA_VAE.py
7 python code\KPIroot.py
8
9 cd ..
10 python agent\run_agent.py --case all --max-reports 4
```

2 附录：提交材料清单

表 21: 最终提交材料清单

材料	说明
GitHub 仓库	https://github.com/Affordan/online-boutique-testing-maintenance
部署文件和部署记录	deploy/、scripts/、docs/02_deployment.md、figures/deployment/
新增微服务源码和部署文件	services/coupon-service/、services/inventory-service/、figures/microservices/
Prometheus 与 Grafana 材料	monitoring/、docs/05_monitoring.md、figures/monitoring/
ChaosMesh 材料	chaos/、docs/06_chaos_experiments.md、figures/chaos/
Selenium 与 JMeter 材料	tests/selenium/、tests/jmeter/、results/testing/、figures/testing/
异常数据集与算法结果	data/raw/、results/monitoring/、algorithms/code/、algorithms/results_DAVAE/、algorithms/result_KPIroot0/、figures/algorithm/
智能运维 Agent	agent/、results/agent/、figures/agent/
大作业报告与展示 PPT	pdf/、pptx/